



Universidad Nacional de Ingeniería

Facultad de Ingeniería Eléctrica y Electrónica

Proyecto Final para el curso de Programación

Orientada a Objetos



BowserSecurity: Detección Inteligente de Armas

Autores:

- | | |
|--------------------------------|-----------|
| - Flores Quiliche Fernando | 20247516A |
| - Palomino Guzmán Bryan Raúl | 20240549A |
| - Mejia Estrada Angel Fernando | 20242161K |

Supervisor: Yury Oscar Tello

Lima, 27 de Junio del 2025

Introduccion

En el contexto actual del Perú, la seguridad ciudadana se ha convertido en una de las principales preocupaciones de la población. El incremento de actos delictivos, como robos a viviendas, asaltos a mano armada e intrusiones en propiedades privadas, ha generado un sentimiento generalizado de vulnerabilidad, especialmente en zonas urbanas y periféricas donde la presencia policial resulta limitada o ineficiente. Esta situación ha impulsado a muchas familias y pequeños negocios a buscar soluciones tecnológicas para proteger sus espacios; sin embargo, gran parte de las alternativas en el mercado se centran en la simple grabación de video o en sistemas de vigilancia que requieren inversiones elevadas y el pago de servicios en la nube. En respuesta a esta realidad, nace BowserSecurity, un sistema de videovigilancia inteligente, económico y autónomo, desarrollado con tecnologías de Internet de las Cosas (IoT) y visión computacional, que busca prevenir antes que reaccionar.

A diferencia de las soluciones convencionales, BowserSecurity está diseñado no solo para captar imágenes, sino para interpretarlas en tiempo real y tomar decisiones automatizadas cuando se detectan comportamientos sospechosos o acciones potencialmente peligrosas. El corazón del sistema es una placa Raspberry Pi, que funciona como unidad de procesamiento local, gestionando una cámara de video conectada y ejecutando un modelo de inteligencia artificial optimizado para la detección de amenazas. Para lograrlo, se ha entrenado un modelo basado en el algoritmo YOLO (You Only Look Once), adaptado para reconocer no solo objetos, sino también acciones específicas como movimientos inusuales, ingreso no autorizado o desplazamientos anómalos dentro de una zona protegida. Esta capacidad de análisis contextual convierte a BowserSecurity en una herramienta preventiva eficaz, que puede lanzar alertas en tiempo real y permitir respuestas inmediatas frente a potenciales riesgos.

La parte lógica del sistema ha sido desarrollada en Python, aplicando la Programación Orientada a Objetos (POO) para estructurar el código de manera clara, modular y fácilmente escalable. Este enfoque permite integrar funcionalidades adicionales sin comprometer la estabilidad del sistema, tales como notificaciones automáticas, grabación de eventos críticos, autenticación de usuarios o incluso el control remoto a través de una interfaz gráfica o móvil. Al trabajar con tecnologías abiertas y accesibles, se garantiza que cualquier persona con conocimientos básicos pueda adaptar o extender el sistema según sus necesidades específicas.

Objetivos:

El proyecto BrowserSecurity tiene como propósito principal el diseño e implementación de un sistema inteligente de vigilancia capaz de detectar amenazas y comportamientos sospechosos en tiempo real, empleando tecnologías de Internet de las Cosas (IoT), visión por computadora e inteligencia artificial, con el fin de ofrecer una solución accesible, autónoma y adaptada a la problemática de seguridad actual.

Objetivo General:

Desarrollar un sistema de videovigilancia inteligente, basado en una placa Raspberry Pi, visión computacional y un modelo de inteligencia artificial entrenado, que permita identificar acciones sospechosas y emitir alertas automáticas en tiempo real, utilizando un enfoque de programación modular con Python y principios de Programación Orientada a Objetos (POO).

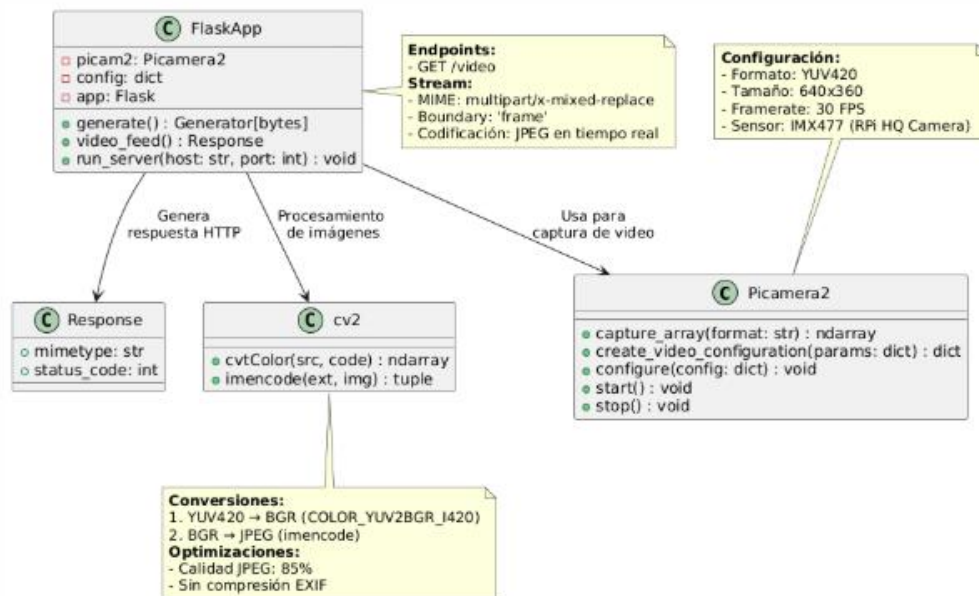
Objetivos Específicos:

- Implementar una arquitectura de hardware utilizando una placa Raspberry Pi conectada a una cámara de videovigilancia, que funcione como unidad de monitoreo autónoma.
- Desarrollar un sistema de software en Python, utilizando Programación Orientada a Objetos (POO) para garantizar escalabilidad, mantenimiento y reutilización del código.
- Entrenar e integrar un modelo de detección basado en YOLO (You Only Look Once), optimizado para el reconocimiento de acciones específicas y amenazas dentro del entorno vigilado.
- Configurar el sistema para procesar video en tiempo real de forma local, permitiendo la identificación inmediata de comportamientos sospechosos y la emisión automática de alertas sin necesidad de intervención humana.
- Diseñar una lógica de control que permita registrar eventos detectados y, opcionalmente, activar módulos externos como alarmas o sistemas de notificación.
- Validar el funcionamiento del sistema en escenarios reales, simulando condiciones de vigilancia en espacios residenciales o comerciales, para evaluar su eficacia y eficiencia frente a sistemas convencionales.

Diagrama UML

En este caso estamos utilizando 2 codigos, por lo tanto se presentaran 2 diagramas en UML, uno que es el que controla la aplicación y la otra que la de la placa Raspberry.

UML 1: Que es el implementado en la placa raspberry para la transmision del video



Subsistema de Transmisión de Video en Tiempo Real

Este grafico UML describe el subsistema del sistema BowserSecurity FIEE encargado de capturar video desde una Raspberry Pi y transmitirlo a través de HTTP utilizando Flask. Este módulo es esencial para alimentar el sistema principal con frames en tiempo real.

Componentes del Subsistema

- FlaskApp: Controlador del servidor.
- Picamera2: Controlador de hardware.
- cv2: Conversión de imágenes.
- Response: Respuesta HTTP de tipo stream.

- Endpoint /video: Punto de entrada del flujo MJPEG.

Clase FlaskApp

Atributos

- picam2, config, app

Métodos

- generate()
- video_feed()
- run_server()

Endpoint /video

- MIME: multipart/x-mixed-replace
- Boundary: -frame
- Codificación: JPEG en tiempo real

Clase Picamera2

- Sensor: IMX477 HQ Camera
- Formato: YUV420, Resolución: 640x360, FPS: 30
- Métodos: capture_array(), start(), stop()

Procesamiento de Imágenes

Funciones

- cv2.cvtColor(): YUV420 a BGR
- cv2.imencode(): BGR a JPEG

Optimización

- Calidad JPEG: 85%
- Sin EXIF

Clase Response

- mimetype = multipart/x-mixed-replace
- status_code = 200

Flujo de Datos

1. Se inicia servidor Flask.

2. Cliente accede a /video.
3. Se genera el stream JPEG.
4. La GUI o navegador lo consume.

Integración

El sistema principal accede a este flujo mediante la URL declarada en GlobalConfig.CAMERA_URL.

Conclusión

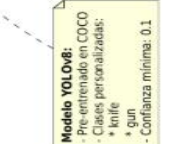
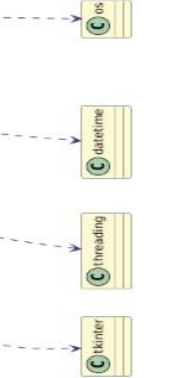
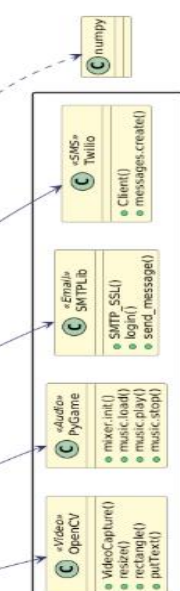
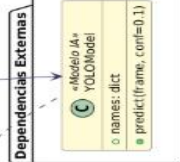
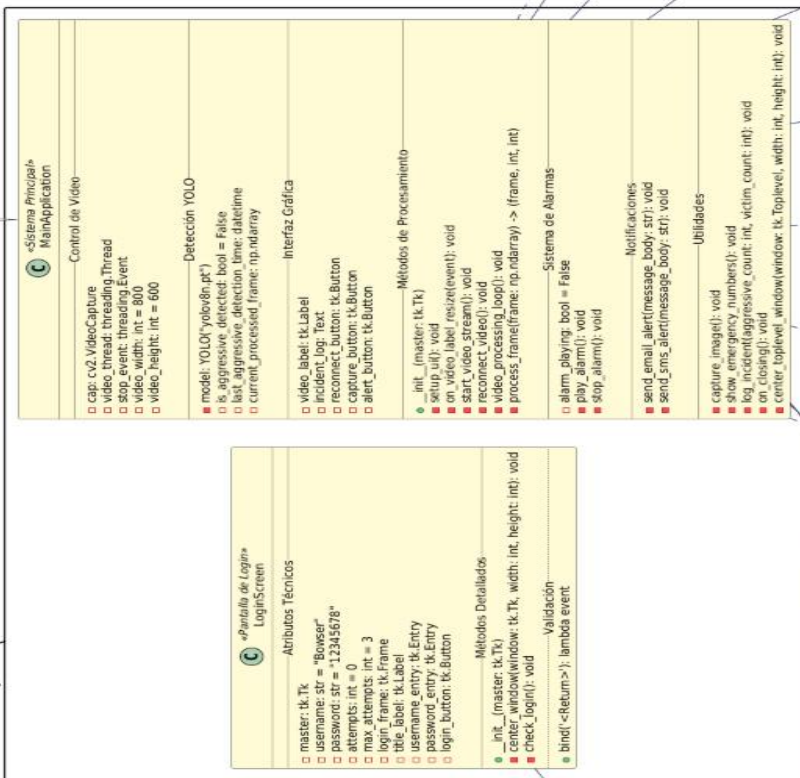
El subsistema provee una transmisión de video en tiempo real confiable, eficiente y modular. Constituye una base sólida para sistemas de monitoreo embebido y se integra fácilmente en arquitecturas más grandes como BrowserSecurity FIEE.

UML 2: Arquitectura UML del Sistema BrowserSecurity FIEE (aplicación)

Flujo de Detección:

1. Captura frame con OpenCV
2. Procesa con YOLOv8n.pt
3. Clasifica personas (agresor/víctima)
4. Si detecta arma:
 - Activa alarma sonora
 - Envía notificaciones
 - Registra incidente
5. Actualiza interfaz gráfica

BrowserSecurity FIEE



Descripción General del Sistema

El sistema ha sido diseñado de forma modular, separando responsabilidades por clases e integrando tecnologías como OpenCV, Tkinter, threading, Pygame, SMTPLib y Twilio. Los módulos principales incluyen:

- **LoginScreen:** Verifica credenciales del usuario.
- **MainApplication:** Núcleo funcional del sistema.
- **GlobalConfig:** Centraliza constantes y credenciales.
- **GlobalFunctions:** Métodos auxiliares reutilizables.
- **YOLOv8Model:** Motor de detección de objetos/personas.

Clase LoginScreen

Propósito

Proporciona control de acceso básico al sistema mediante verificación de usuario y contraseña.

Atributos

- username = "Bowser"
- password = "12345678"
- attempts = 0, max_attempts = 3
- Widgets: username_entry, password_entry, login_button

Métodos

- check_login(): Valida credenciales.
- center_window(): Centra la ventana.
- Evento bind("<Return>"): Permite usar ENTER para validar.

Clase MainApplication

Propósito

Es el componente principal que administra el flujo de video, procesamiento de imágenes, interfaz gráfica, activación de alarmas y envío de alertas.

Módulos Internos

Control de Video

- `cv2.VideoCapture`: Captura de cámara.
- Resolución: 800x600.
- Hilos: `video_thread`, `stop_event`.

YOLOv8 y Procesamiento

- Modelo cargado: `yolov8n.pt`.
- Clases detectadas: "knife", "gun".
- Método: `process_frame()`.

Interfaz Gráfica

- Widgets: `video_label`, `incident_log`.
- Botones: `reconnect_button`, `capture_button`, `alert_button`.

Sistema de Alarmas

- `play_alarm()`, `stop_alarm()` con `pygame`.

Notificaciones

- `send_email_alert()`, `send_sms_alert()`.

Ciclo de Vida

- `start_video_stream()`
- `video_processing_loop()`
- `on_closing()`

Clase GlobalConfig

Constantes Definidas

- `AGGRESSIVE_CLASSES = ['knife', 'gun']`
- `CAMERA_URL = "https://raspberrypi.ngrok.app/video"`
- `ALARM_SOUND_PATH = "alarm.mp3"`

Credenciales

- `EMAIL_SENDER`, `EMAIL_PASSWORD`
- `TWILIO_ACCOUNT_SID`, `TWILIO_AUTH_TOKEN`

- **Contacto Emergencia:**
 - Correo: flores.q@uni.pe
 - SMS: +51917954631

Clase GlobalFunctions

Métodos

- send_login_email()
- send_login_sms()

Modelo YOLOv8

Especificaciones

- Modelo ligero YOLOv8n.
- Pre-entrenado con COCO + personalización.
- Confianza mínima: 0.1.

Método

- predict(frame): retorna etiquetas y coordenadas.

Flujo de Detección

5. Se captura un frame del video.
6. El frame se envía al modelo YOLOv8.
7. Se compara contra las clases agresivas.
8. Si hay coincidencia:
 - Se activa alarma sonora.
 - Se envía alerta por email y SMS.
 - Se registra evento en log.
 - Se actualiza GUI.

Dependencias Externas

- **Tkinter:** GUI principal.
- **OpenCV:** Captura y visualización de video.
- **threading:** Manejo de concurrencia.
- **pygame:** Reproducción de sonido de alarma.

- **smtplib:** Envío de correos.
- **twilio:** Envío de SMS.
- **datetime:** Registro de eventos.
- **numpy:** Procesamiento matricial.

Seguridad

- **Advertencia:** Las credenciales están en texto plano.
- **Recomendación:** Usar archivos .env y cargarlos con la librería dotenv.
- **Login:** No incluye cifrado ni verificación multifactor.

Integración entre los Subsistemas Flask + Picamera2 y BowserSecurity FIEE

El sistema de seguridad inteligente **BowserSecurity FIEE** presenta una arquitectura modular compuesta por subsistemas especializados que interactúan entre sí a través de interfaces bien definidas. Este informe documenta con profundidad la integración entre:

- El subsistema de captura y transmisión de video en tiempo real basado en **Flask + Picamera2**.
- El subsistema principal **BowserSecurity FIEE**, encargado del procesamiento de video, detección de amenazas mediante YOLOv8, activación de alarmas y notificación.

Ambos subsistemas están diseñados para funcionar de forma **desacoplada**, comunicándose a través de la red mediante el protocolo HTTP. Esta separación ofrece portabilidad, escalabilidad y facilidad de mantenimiento.

Resumen de los Subsistemas

Subsistema 1: Flask + Picamera2

Este subsistema es responsable de capturar video desde una cámara Raspberry Pi HQ (sensor IMX477) y servirlo a través de un endpoint HTTP utilizando el formato multipart/x-mixed-replace con codificación JPEG.

- **Clase principal:** FlaskApp
- **Cámara:** Picamera2

- **Formato de video:** YUV420 → BGR → JPEG
- **Endpoint:** /video
- **Puerto:** configurable (por defecto 5000)

Subsistema 2: BowserSecurity FIEE

Este es el sistema principal que recibe los frames de video, los procesa con YOLOv8 para detectar amenazas (armas, agresiones), activa alarmas sonoras, registra eventos y envía notificaciones por correo y SMS.

- **Clase principal:** MainApplication
- **Interfaz:** Gráfica con Tkinter
- **Modelo de detección:** YOLOv8 entrenado con clases personalizadas (knife, gun)
- **Canal de entrada de video:** URL HTTP configurada en GlobalConfig.CAMERA_URL

Paso a Paso de la Integración

Paso 1: Inicio del servidor Flask

El servidor se inicia mediante el método:

```
FlaskApp.run_server(host='0.0.0.0', port=5000)
```

Esto permite exponer el endpoint /video accesible desde cualquier cliente en la red local o a través de un ngrok tunnel configurado.

Paso 2: Captura y codificación de video

9. Se crea una instancia de Picamera2.
10. Se configura con resolución 640x360, 30 FPS y formato YUV420.
11. En cada iteración del generador generate(), se captura un frame:


```
frame = picam2.capture_array("main")
```
12. Se convierte de YUV420 a BGR:


```
frame_bgr = cv2.cvtColor(frame, cv2.COLOR_YUV2BGR_I420)
```
13. Se codifica como JPEG:


```
ret, jpeg = cv2.imencode(".jpg", frame_bgr, [int(cv2.IMWRITE_JPEG_QUALITY), 85])
```
14. Se envía el resultado como parte del stream HTTP con el boundary -frame.

Paso 3: Consumo del stream en el sistema principal

- En la clase MainApplication, se crea un objeto de captura OpenCV:

```
cap = cv2.VideoCapture(GlobalConfig.CAMERA_URL)
```

- Esta URL apunta al endpoint del servidor Flask, por ejemplo:

```
http://raspberrypi.local:5000/video
```

- A partir de ese momento, MainApplication recibe en tiempo real los frames JPEG y puede procesarlos.

Paso 4: Análisis con YOLOv8

Cada frame recibido se analiza con el modelo YOLOv8:

```
results = model.predict(frame)
```

Si se detectan clases definidas como agresivas (por ejemplo gun, knife), se ejecuta el siguiente flujo:

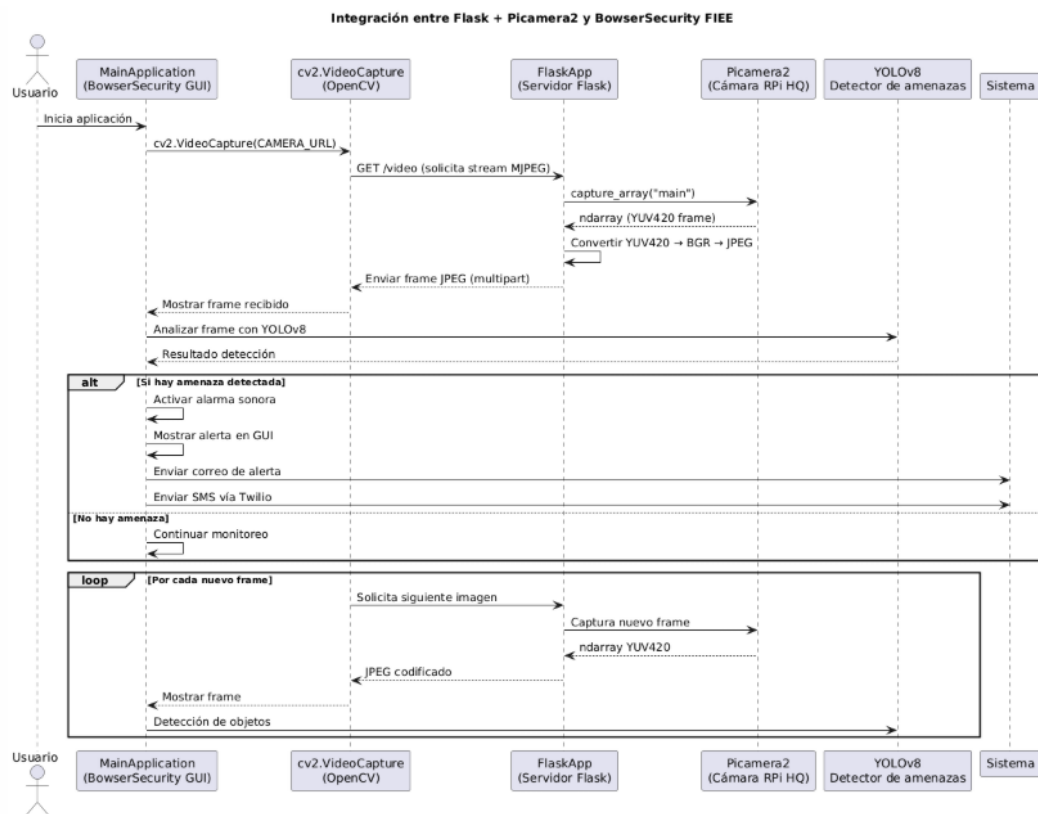
1. Activación de alarma sonora (PyGame): `play_alarm()`
2. Envío de correo (SMTP): `send_email_alert()`
3. Envío de SMS (Twilio): `send_sms_alert()`
4. Registro visual en GUI (Tkinter)

Paso 5: Ciclo continuo de monitoreo

El sistema está preparado para ejecutar este ciclo continuamente:

1. Captura de frame del stream HTTP
2. Análisis con modelo YOLO
3. Verificación de amenazas
4. Activación de alarmas y notificación
5. Repetición del proceso

Flujo de Comunicación entre Subsistemas



Ventajas del Modelo Desacoplado

- **Modularidad:** Los subsistemas pueden probarse y actualizarse de forma independiente.
- **Escalabilidad:** Se puede cambiar la fuente de video sin tocar el sistema principal.
- **Despliegue flexible:** El servidor Flask puede estar en otro dispositivo (otra Raspberry Pi o servidor remoto).
- **Seguridad:** Permite insertar firewalls, autenticación o tuneles seguros entre ambos componentes.

Consideraciones de Seguridad y Mejora

- Incorporar autenticación básica o token en el endpoint /video
- Configurar HTTPS con certificado SSL (Flask + ssl_context)
- Agregar redundancia en el cliente para intentar reconexiones automáticas
- Monitorear pérdida de paquetes o calidad de transmisión

La integración entre Flask + Picamera2 y el sistema principal BrowserSecurity FIEE es un ejemplo claro de arquitectura desacoplada y eficiente. Aprovechando HTTP como canal de transmisión, permite dividir la responsabilidad de captura y procesamiento, logrando un sistema robusto, extensible y escalable. Esta estructura permite futuras expansiones como el uso de otras cámaras, la integración en la nube o el control desde interfaces web remotas.

Explicación del código de la aplicación en Python:

A continuación, se explicara el código empleado en Python para la aplicación, haciendo uso de conjuntos de líneas para una mejor visualización. Las líneas en blanco han sido omitidas.

```
C:\Users\ASUS > OneDrive > Escritorio > Seguridad (renovado).py > ...
1  import tkinter as tk
2  from tkinter import ttk, messagebox, filedialog
3  from datetime import datetime
4  import time
5  import threading
6  import shutil
7  import cv2
8  from PIL import Image, ImageTk
9  import socket
10 import os
11 import logging
12 from ultralytics import YOLO
13 import winsound # Para reproducir sonido en Windows
14 from twilio.rest import Client # Para enviar SMS con Twilio
15
16 # Nuevas importaciones para el envío de correo:
17 from dotenv import load_dotenv
18 import smtplib
19 import ssl
20 from email.mime.multipart import MIMEMultipart
21 from email.mime.text import MIMEText
22
23 # Cargar variables de entorno desde .env (si las utilizas):
24 load_dotenv()
25
26 # ----- CONFIGURACIÓN DE LOGGING PARA AUTENTICACIÓN -----
27 logger_auth = logging.getLogger("auth_logger")
28 logger_auth.setLevel(logging.INFO)
29 fh = logging.FileHandler("auth.log", encoding="utf-8")
30 fh.setLevel(logging.INFO)
31 formatter = logging.Formatter("%(asctime)s - %(message)s", datefmt="%Y-%m-%d %H:%M:%S")
32 fh.setFormatter(formatter)
33 logger_auth.addHandler(fh)
34 # -----
```

Líneas 1-34:

Línea 1: import tkinter as tk

- **tkinter**: Biblioteca estándar de Python para crear interfaces gráficas de usuario (GUI).
- **as tk**: Se asigna un alias para usar tk.Label, tk.Button, etc., en lugar de tkinter.Label, etc.

Línea 2: from tkinter import ttk, messagebox, filedialog

- **ttk**: Submódulo de tkinter con widgets mejorados (estilo moderno).
- **messagebox**: Para mostrar cuadros de diálogo de alerta, información, error, etc.
- **filedialog**: Para abrir ventanas de selección de archivos o carpetas.

Linea 3: from datetime import datetime

- Permite trabajar con fechas y horas actuales (datetime.now()), convertir formatos, etc.

Linea 4: import time

- Proporciona funciones para manejar tiempos de espera (sleep), medir tiempo, etc.

Linea 5: import threading

- Permite ejecutar **tareas en segundo plano** sin bloquear la interfaz gráfica.

Linea 6: import shutil

- Sirve para copiar, mover, eliminar archivos y manipular estructuras de directorios.

Linea 7: import cv2

- Importa OpenCV, biblioteca de visión artificial. Se usa para procesar imágenes, capturar video, etc.

Linea 8: from PIL import Image, ImageTk

- PIL (Pillow): Librería para trabajar con imágenes.
- Image: Para abrir, modificar, guardar imágenes.
- ImageTk: Para convertir imágenes en formato compatible con tkinter.

Linea 9: import socket

- Permite comunicaciones de red (por ejemplo, obtener IPs, hacer servidores, etc.).

Linea 10: import os

- Acceso al sistema operativo: rutas, variables de entorno, archivos, procesos.

Linea 11: import logging

- Módulo estándar para crear registros de eventos (logs). Útil para depuración y seguimiento.

Linea 12: from ultralytics import YOLO

- Importa el modelo **YOLO (You Only Look Once)** desde la librería ultralytics para realizar detección de objetos en tiempo real.

Linea 13: import winsound # Para reproducir sonido en Windows

- Permite emitir sonidos en sistemas Windows (ej. una alarma al detectar algo).

Linea 14: from twilio.rest import Client # Para enviar SMS con Twilio

- Cliente oficial de la API de Twilio para enviar mensajes de texto, llamadas, etc.

Linea 17: from dotenv import load_dotenv

- Carga variables desde un archivo .env (por ejemplo, usuario, clave, API keys) para mayor seguridad.

Linea 18: import smtplib

- Librería para enviar correos usando el protocolo SMTP (Simple Mail Transfer Protocol).

Linea 19: import ssl

- Proporciona soporte para conexiones cifradas (TLS/SSL), usadas en envío seguro de emails.

Linea 20: from email.mime.multipart import MIMEMultipart

Linea 21: from email.mime.text import MIMEText

- Permiten componer correos con texto plano o HTML.
- MIMEMultipart: Cuerpo del correo que puede tener varias partes (texto + archivos).
- MIMEText: Define el contenido del mensaje (texto o HTML).

Linea 27: logger_auth = logging.getLogger("auth_logger")

- Crea un **logger** personalizado llamado "auth_logger" para registrar eventos relacionados con la autenticación.

Linea 28: logger_auth.setLevel(logging.INFO)

- Establece que el nivel mínimo de mensajes a registrar será INFO (no registra DEBUG, sí INFO, WARNING, ERROR...).

Linea 29: fh = logging.FileHandler("auth.log", encoding="utf-8")

- Crea un **manejador de archivo** (FileHandler) para guardar los logs en un archivo llamado auth.log.

Linea 30: fh.setLevel(logging.INFO)

- Define que este archivo recibirá mensajes desde nivel INFO hacia arriba.

Linea 31: formatter = logging.Formatter("%(asctime)s - %(message)s", datefmt="%Y-%m-%d %H:%M:%S")

- Establece el **formato** del mensaje del log. Incluirá:
 - %(asctime)s: Fecha y hora del evento
 - %(message)s: Contenido del mensaje

Linea 32: fh.setFormatter(formatter)

- Aplica el formato definido al manejador de archivo.

Linea 33: logger_auth.addHandler(fh)

- Asocia el manejador al logger personalizado, de modo que cualquier mensaje enviado a logger_auth se escriba en auth.log.

```

35
36 def obtener_ip_local():
37     try:
38         s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
39         s.connect(("8.8.8.8", 80))
40         ip_local = s.getsockname()[0]
41         s.close()
42         return ip_local
43     except Exception:
44         return "127.0.0.1"
45
46 class LoginApp:
47     def __init__(self):
48         self.login_window = tk.Tk()
49         self.login_window.title("Login - Sistema de Seguridad")
50         self.login_window.geometry("400x300")
51         self.login_window.configure(bg="#1E1E2F")
52         self.login_window.resizable(False, False)
53
54         self.failed_attempts = 0
55
56         style_login = ttk.Style()
57         style_login.theme_use('clam')
58         style_login.configure("TLabel", background="#1E1E2F", foreground="#E0E0E0", font=("Arial", 12))
59         style_login.configure("TEntry", fieldbackground="#2A2A3D", foreground="white", font=("Arial", 12))
60         style_login.configure("TButton", padding=10, relief="flat", background="#8A8A9E2", foreground="white",
61                               font=("Arial", 12, "bold"))
62
63         main_frame = ttk.Frame(self.login_window, padding=20)
64         main_frame.place(relx=0.5, rely=0.5, anchor=tk.CENTER)
65
66         ttk.Label(main_frame, text="Usuario:").grid(row=0, column=0, sticky='w', pady=10)
67         self.user_entry = tk.Entry(main_frame, width=30)
68         self.user_entry.grid(row=0, column=1)
69
70         ttk.Label(main_frame, text="Contraseña:").grid(row=1, column=0, sticky='w', pady=10)

```

Lineas 36-69:

Linea 36: def obtener_ip_local():

- Define una función llamada obtener_ip_local que retorna la dirección IP local del equipo.

Linea 37: try:

- Inicia un bloque try para capturar errores potenciales durante la obtención de la IP.

Linea 38: s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

- Crea un socket de red (s) usando IPv4 (AF_INET) y protocolo UDP (SOCK_DGRAM).

Linea 39: s.connect(("8.8.8.8", 80))

- Conecta el socket a la IP pública de Google (8.8.8.8) en el puerto 80; no envía datos, solo fuerza al SO a elegir la interfaz de salida.

Linea 40: ip_local = s.getsockname()[0]

- Obtiene la tupla (IP, puerto) asociada al socket y extrae la IP local asignada ([0]).

Linea 41: s.close()

- Cierra el socket para liberar recursos del sistema.

Linea 42: return ip_local

- Devuelve la dirección IP local obtenida.

Linea 43: except Exception:

- Captura cualquier excepción que ocurra dentro del bloque try.

Linea 44: return "127.0.0.1"

- Si hay un error (por ejemplo, sin conexión a red), devuelve la IP de loopback como fallback.

Linea 46: class LoginApp:

- Define la clase LoginApp, responsable de la ventana de inicio de sesión del sistema.

Linea 47: def __init__(self):

- Método constructor que inicializa todos los componentes de la interfaz de login.

Linea 48: self.login_window = tk.Tk()

- Crea la ventana principal de tkinter y la asigna a self.login_window.

Linea 49: self.login_window.title("Login - Sistema de Seguridad")

- Establece el título de la ventana como "Login – Sistema de Seguridad".

Linea 50: self.login_window.geometry("400x300")

- Define el tamaño inicial de la ventana a 400×300 píxeles.

Linea 51: self.login_window.configure(bg="#1E1E2F")

- Configura el color de fondo de la ventana (un gris oscuro).

Linea 52: self.login_window.resizable(False, False)

- Deshabilita la opción de redimensionar la ventana en ancho y alto.

Linea 54: self.failed_attempts = 0

- Inicializa un contador de intentos fallidos de login en cero.

Linea 56: style_login = ttk.Style()

- Crea un objeto Style para personalizar los estilos de los widgets ttk.

Linea 57: style_login.theme_use('clam')

- Aplica el tema "clam" (neutral y moderno) a los widgets.

Linea 58: style_login.configure("TLabel", background="#1E1E2F", foreground="#E0E0E0", font=("Arial", 12))

- Establece estilo para etiquetas: fondo oscuro, texto claro y fuente Arial 12.

Linea 59: style_login.configure("TEntry", fieldbackground="#2A2A3D", foreground="white", font=("Arial", 12))

- Define estilo para campos de entrada: color de fondo medio oscuro y texto blanco.

Línea 60: style_login.configure("TButton", padding=10, relief="flat", background="#4A90E2", foreground="white",
Línea 61: font=("Arial", 12, "bold"))

- Configura botones con 10 px de relleno, borde plano, azul de fondo, texto blanco y fuente negra.

Línea 63: main_frame = ttk.Frame(self.login_window, padding=20)

- Crea un contenedor (Frame) con 20 px de padding en la ventana de login.

Línea 64: main_frame.place(relx=0.5, rely=0.5, anchor=tk.CENTER)

- Posiciona el main_frame centrado en la ventana mediante coordenadas relativas.

Línea 66: ttk.Label(main_frame, text="Usuario:").grid(row=0, column=0, sticky='w', pady=10)

- Inserta una etiqueta "Usuario:" en la fila 0, columna 0, alineada al oeste, con espaciado vertical.

Línea 67: self.user_entry = ttk.Entry(main_frame, width=30)

- Crea el campo de texto para ingresar el usuario (30 caracteres de ancho).

Línea 68: self.user_entry.grid(row=0, column=1)

- Coloca el Entry de usuario en la fila 0, columna 1 del main_frame.

```
70     ttk.Label(main_frame, text="Contraseña:").grid(row=1, column=0, sticky='w', pady=10)
71     self.pass_entry = ttk.Entry(main_frame, width=30, show="*")
72     self.pass_entry.grid(row=1, column=1)
73
74     self.login_button = ttk.Button(main_frame, text="Acceder", command=self.verificar_login)
75     self.login_button.grid(row=2, column=0, columnspan=2, pady=20)
76
77     self.segundos_restantes = 0
78     self.contador_label = ttk.Label(main_frame, text="", font=("Arial", 10), foreground="#F5A623")
79     self.contador_label.grid(row=3, column=0, columnspan=2)
80
81     def verificar_login(self):
82         user = self.user_entry.get()
83         pwd = self.pass_entry.get()
84
85         if self.failed_attempts >= 3:
86             messagebox.showwarning("Bloqueado", "Demasiados intentos fallidos. Intenta de nuevo más tarde.")
87             return
88
89         if user == "Bowser" and pwd == "12345678":
90             logger_auth.info(f'LOGIN SUCCESS user="{user}"')
91             self.login_window.destroy()
92             app = SeguridadApp()
93             app.iniciar_app()
94         else:
95             logger_auth.info(f'LOGIN FAIL user="{user}"')
96             self.failed_attempts += 1
97             remaining = max(0, 3 - self.failed_attempts)
98             messagebox.showerror("Error", f"Usuario o contraseña incorrectos.\nIntentos restantes: {remaining}")
99
100         if self.failed_attempts >= 3:
101             self.login_button.config(state="disabled")
102             self.segundos_restantes = 30
103             self.actualizar_contador()
104             self.login_window.after(30000, self.reactivar_login)
```

Lineas 70-104:

Línea 70: ttk.Label(main_frame, text="Contraseña:").grid(row=1, column=0, sticky='w', pady=10)

- Inserta una etiqueta “Contraseña:” en main_frame, fila 1 columna 0.
- sticky='w' alinea el texto a la izquierda; pady=10 agrega espacio vertical.

Línea 71: self.pass_entry = ttk.Entry(main_frame, width=30, show="*")

- Crea un campo de entrada para la contraseña de 30 caracteres de ancho.
- show="*" oculta los caracteres ingresados mostrando asteriscos.

Línea 72: self.pass_entry.grid(row=1, column=1)

- Coloca el Entry de contraseña en la fila 1, columna 1 del main_frame.

Línea 74: self.login_button = ttk.Button(main_frame, text="Acceder", command=self.verificar_login)

- Crea un botón “Acceder” que, al pulsarlo, invoca el método self.verificar_login.

Línea 75: self.login_button.grid(row=2, column=0, columnspan=2, pady=20)

- Ubica el botón en la fila 2 abarcando dos columnas, con 20 px de espacio vertical.

Línea 77: self.segundos_restantes = 0

- Inicializa en cero el contador de segundos que queda bloqueado tras 3 intentos fallidos.

Línea 78: self.contador_label = ttk.Label(main_frame, text="", font=("Arial", 10), foreground="#F5A623")

- Crea una etiqueta vacía para mostrar la cuenta regresiva de bloqueo.
- Usa fuente Arial 10 y texto color naranja claro (#F5A623).

Línea 79: self.contador_label.grid(row=3, column=0, columnspan=2)

- Coloca la etiqueta de contador en la fila 3, cubriendo ambas columnas.

Línea 81: def verificar_login(self):

- Define el método que valida usuario y contraseña al pulsar “Acceder”.

Línea 82: user = self.user_entry.get()

- Obtiene el texto ingresado en el campo de usuario.

Línea 83: pwd = self.pass_entry.get()

- Obtiene el texto (oculto) ingresado en el campo de contraseña.

Línea 85: if self.failed_attempts >= 3:

- Verifica si ya hubo tres o más intentos fallidos seguidos.

Línea 86: messagebox.showwarning("Bloqueado", "Demasiados intentos fallidos. Intenta de nuevo más tarde.")

- Muestra un cuadro de advertencia indicando bloqueo temporal.

Línea 87: return

- Sale del método sin procesar más validaciones.

Línea 89: if user == "Bowser" and pwd == "12345678":

- Comprueba credenciales: usuario "Bowser" y contraseña "12345678".

Línea 90: logger_auth.info(f'LOGIN SUCCESS user="{user}")

- Registra en el log de autenticación un mensaje de éxito.

Línea 91: self.login_window.destroy()

- Cierra la ventana de login.

Línea 92: app = SeguridadApp()

- Instancia la clase principal de la aplicación de seguridad.

Línea 93: app.iniciar_app()

- Lanza el bucle principal (mainloop) de la interfaz de seguridad.

Línea 94: else:

Si las credenciales no coinciden, ejecuta el bloque de falla.

Línea 95: logger_auth.info(f'LOGIN FAIL user="{user}")

- Registra en el log un mensaje de intento fallido.

Línea 96: self.failed_attempts += 1

- Incrementa el contador de intentos fallidos.

Línea 97: remaining = max(0, 3 - self.failed_attempts)

- Calcula cuántos intentos quedan antes del bloqueo (no negativo).

Línea 98: messagebox.showerror("Error", f"Usuario o contraseña incorrectos.\nIntentos restantes: {remaining}")

- Muestra un cuadro de error con los intentos que quedan.

Línea 100: if self.failed_attempts >= 3:

- Si tras este fallo se llega a 3 intentos, activa el bloqueo.

Línea 101: self.login_button.config(state="disabled")

- Deshabilita el botón de acceso para evitar más clics.

Línea 102: self.segundos_restantes = 30

- Establece 30 s de bloqueo antes de reactivar el login.

Línea 103: self.actualizar_contador()

- Inicia la función que actualiza cada segundo la etiqueta del contador.

Línea 104: `self.login_window.after(30000, self.reactivar_login)`

```

104 | self.login_window.after(30000, self.reactivar_login)
105 |
106 | def actualizar_contador(self):
107 |     if self.segundos_restantes > 0:
108 |         self.contador_label.config(text=f"Bloqueado. Intenta en {self.segundos_restantes}s")
109 |         self.segundos_restantes -= 1
110 |         self.login_window.after(1000, self.actualizar_contador)
111 |
112 | def reactivar_login(self):
113 |     self.failed_attempts = 0
114 |     self.login_button.config(state="normal")
115 |     self.contador_label.config(text="")
116 |     messagebox.showinfo("Reactivado", "Ahora puedes intentar iniciar sesión de nuevo.")
117 |
118 | def run(self):
119 |     self.login_window.mainloop()
120 |
121 | class SeguridadApp:
122 |     def __init__(self):
123 |         self.dark_mode = True
124 |         self.live_activado = False
125 |         self.cap = None
126 |         self.hilo_live = None
127 |         self.imagen_capturada_path = None
128 |         self.model = YOLO("yolov8n.pt")
129 |         self.ultimos_objetos_detectados = [] # Almacenar los últimos objetos detectados
130 |
131 |         # Configuración de Twilio para enviar SMS
132 |         self.twilio_account_sid = "AC0d739fd805cc024dbba19958dc5fa1f3"
133 |         self.twilio_auth_token = "cde421f7afe63733ce76378b708e7ffe"
134 |         self.twilio_phone_number = "+18635922506"
135 |         self.destinatarios_phone_numbers = ["+51940317018", "+51925446899"]
136 |         self.twilio_client = Client(self.twilio_account_sid, self.twilio_auth_token)
137 |
138 |         if not os.path.exists("capturas"):

```

Líneas 106-138:

Línea 106: `def actualizar_contador(self):`

- Define el método que actualiza cada segundo la etiqueta de cuenta regresiva durante el bloqueo.

Línea 107: `if self.segundos_restantes > 0:`

- Comprueba si aún quedan segundos por contar antes de reactivar el login.

Línea 108: `self.contador_label.config(text=f"Bloqueado. Intenta en {self.segundos_restantes}s")`

- Actualiza el texto de `contador_label` mostrando cuántos segundos faltan.

Línea 109: `self.segundos_restantes -= 1`

- Decrementa en 1 el valor de `self.segundos_restantes`.

Línea 110: `self.login_window.after(1000, self.actualizar_contador)`

- Programa la siguiente llamada a este método tras 1 000 ms para continuar la cuenta atrás.

Línea 112: `def reactivar_login(self):`

- Define el método que restaura el estado del login tras el periodo de bloqueo.

Línea 113: `self.failed_attempts = 0`

- Reinicia el contador de intentos fallidos a cero.

Linea 114: self.login_button.config(state="normal")

- Vuelve a habilitar el botón "Acceder".

Linea 115: self.contador_label.config(text="")

- Limpia el texto del contador.

Linea 116: messagebox.showinfo("Reactivado", "Ahora puedes intentar iniciar sesión de nuevo.")

- Muestra un cuadro informativo indicando que el login está disponible de nuevo.

Linea 118: def run(self):

- Define el método que inicia el bucle principal de la ventana de login.

Linea 119: self.login_window.mainloop()

- Ejecuta el mainloop() de tkinter, mostrando la interfaz y gestionando eventos.

Linea 121: class SeguridadApp:

- Declara la clase principal de la aplicación de seguridad, que gestiona la cámara, detección y alertas.

Linea 122: def __init__(self):

- Método constructor que inicializa todos los atributos y la interfaz de SeguridadApp.

Linea 123: self.dark_mode = True

- Variable booleana para controlar si la interfaz está en modo oscuro.

Linea 124: self.live_activado = False

- Indica si el vídeo en vivo está activo o no.

Linea 125: self.cap = None

- Referencia al objeto VideoCapture de OpenCV, inicialmente None.

Linea 126: self.hilo_live = None

- Referencia al hilo que procesa el vídeo en vivo, inicialmente None.

Linea 127: self.imagen_capturada_path = None

- Almacena la ruta de la última imagen capturada, inicialmente None.

Linea 128: self.model = YOLO("yolov8n.pt")

- Carga el modelo YOLOv8 (archivo yolov8n.pt) para detección de objetos.

Linea 129: self.ultimos_objetos_detectados = []

- Lista para guardar los nombres de los últimos objetos detectados en un fotograma.

Linea 131: # Configuración de Twilio para enviar SMS

- Comentario que indica el inicio de la sección de configuración de la API de Twilio.

Linea 132: self.twilio_account_sid = "AC0d739fd805cc024dbba19958dc5fa1f3"

- Almacena el SID de la cuenta de Twilio (identificador público).

Linea 133: self.twilio_auth_token = "cde421f7afe63733ce76378b708e7ffe"

- Guarda el token de autenticación de Twilio (secreto).

Linea 134: self.twilio_phone_number = "+18635922506"

- Número de teléfono registrado en Twilio desde el que se enviarán los SMS.

Linea 135: self.destinatarios_phone_numbers = ["+51940317018", "+51925446899"]

- Lista de números de destinatarios a los que se enviarán las alertas por SMS.

Linea 136: self.twilio_client = Client(self.twilio_account_sid, self.twilio_auth_token)

- Crea el cliente de Twilio usando SID y token para enviar mensajes.

Linea 138: if not os.path.exists("capturas"):

- Comprueba si no existe la carpeta capturas en el directorio actual para almacenar imágenes.

```
139 | os.makedirs("capturas", exist_ok=True)
140 |
141 | self.root = tk.Tk()
142 | self.root.title("🔒 Sistema de Seguridad Inteligente")
143 | self.root.geometry("1200x800")
144 | self.root.configure(bg="#1E1E2F")
145 | self.root.protocol("WM_DELETE_WINDOW", self.cerrar_app)
146 |
147 | menubar = tk.Menu(self.root)
148 | file_menu = tk.Menu(menubar, tearoff=0)
149 | file_menu.add_command(label="Salir", command=self.cerrar_app)
150 | menubar.add_cascade(label="Archivo", menu=file_menu)
151 |
152 | view_menu = tk.Menu(menubar, tearoff=0)
153 | self.var_darkmode = tk.BooleanVar(value=self.dark_mode)
154 | view_menu.add_checkbutton(label="Modo Oscuro", onvalue=True, offvalue=False, variable=self.var_darkmode,
155 |                           command=self.toggle_dark_mode)
156 | menubar.add_cascade(label="Ver", menu=view_menu)
157 | self.root.config(menu=menubar)
158 |
159 | self.style = ttk.Style()
160 | self.style.theme_use('clam')
161 | self._configurar_estilos()
162 |
163 | container = ttk.Frame(self.root)
164 | container.pack(fill=tk.BOTH, expand=True, padx=10, pady=10)
165 |
166 | self.video_frame = ttk.Frame(container, width=800, relief=tk.RIDGE, borderwidth=2)
167 | self.video_frame.pack(side=tk.LEFT, fill=tk.BOTH, expand=True, padx=(0, 10))
168 |
169 | right_frame = ttk.Frame(container, width=350)
170 | right_frame.pack(side=tk.RIGHT, fill=tk.Y, expand=False)
171 |
172 | titulo = ttk.Label(self.video_frame, text="🔒 Sistema de Seguridad Inteligente", font=("Arial", 24, "bold"),
173 |                   foreground="#4A90E2")
```

Lineas 139-173:

Linea 139: os.makedirs("capturas", exist_ok=True)

- Crea la carpeta capturas si no existe.
- exist_ok=True evita error si ya existe.

Línea 141: self.root = tk.Tk()

- Crea la ventana principal de la aplicación de seguridad.

Línea 142: self.root.title("🔒 Sistema de Seguridad Inteligente")

- Establece el título de la ventana con un emoji de candado.

Línea 143: self.root.geometry("1200x800")

- Define el tamaño inicial de la ventana a 1200×800 píxeles.

Línea 144: self.root.configure(bg="#1E1E2F")

- Configura el color de fondo de la ventana (gris oscuro).

Línea 145: self.root.protocol("WM_DELETE_WINDOW", self.cerrar_app)

- Intercepta el cierre de ventana para ejecutar self.cerrar_app y liberar recursos.

Línea 147: menubar = tk.Menu(self.root)

- Crea la barra de menús en la ventana principal.

Línea 148: file_menu = tk.Menu(menubar, tearoff=0)

- Crea el menú "Archivo" sin la línea de separación flotante.

Línea 149: file_menu.add_command(label="Salir", command=self.cerrar_app)

- Añade opción "Salir" que invoca self.cerrar_app.

Línea 150: menubar.add_cascade(label="Archivo", menu=file_menu)

- Agrega el menú "Archivo" a la barra de menús.

Línea 152: view_menu = tk.Menu(menubar, tearoff=0)

- Crea el menú "Ver" (para opciones de vista) sin tearoff.

Línea 153: self.var_darkmode = tk.BooleanVar(value=self.dark_mode)

- Variable booleana vinculada al estado de modo oscuro.

Línea 154: view_menu.add_checkbutton(label="Modo Oscuro", onvalue=True,
offvalue=False, variable=self.var_darkmode,

Línea 155: command=self.toggle_dark_mode)

- Añade casilla al menú "Ver" para activar/desactivar modo oscuro.
- Cambia self.var_darkmode y llama a self.toggle_dark_mode.

Línea 156: menubar.add_cascade(label="Ver", menu=view_menu)

- Agrega el menú "Ver" a la barra de menús.

Línea 157: self.root.config(menu=menubar)

- Asocia la barra de menús (menubar) a la ventana.

Linea 159: `self.style = ttk.Style()`

- Crea un objeto Style para personalizar estilos en toda la app.

Linea 160: `self.style.theme_use('clam')`

- Aplica el tema “clam” a los widgets ttk.

Linea 161: `self._configurar_estilos()`

- Llama al método que ajusta colores y fuentes según el modo oscuro.

Linea 163: `container = ttk.Frame(self.root)`

- Crea un contenedor principal para organizar los paneles.

Linea 164: `container.pack(fill=tk.BOTH, expand=True, padx=10, pady=10)`

- Empaqueta el contenedor para llenar la ventana con padding de 10 px.

Linea 166: `self.video_frame = ttk.Frame(container, width=800, relief=tk.RIDGE, borderwidth=2)`

- Crea un marco para mostrar el video con borde en relieve.

Linea 167: `self.video_frame.pack(side=tk.LEFT, fill=tk.BOTH, expand=True, padx=(0, 10))`

- Empaqueta video_frame a la izquierda, llenando espacio y dejando 10 px a la derecha.

Linea 169: `right_frame = ttk.Frame(container, width=350)`

- Crea el panel derecho para controles y registros.

Linea 170: `right_frame.pack(side=tk.RIGHT, fill=tk.Y, expand=False)`

- Empaqueta right_frame a la derecha, llenando verticalmente.

Linea 172: `titulo = ttk.Label(self.video_frame, text="🛡 Sistema de Seguridad Inteligente", font=("Arial", 24, "bold"),`

Linea 173: `foreground="#4A90E2")`

- Crea una etiqueta grande con título y color azul para el encabezado del sistema.

```

174 titulo.pack(pady=10)
175
176 self.imagen_label = ttk.Label(self.video_frame, borderwidth=2, relief="flat", bg=" #2A2A3D")
177 self.imagen_label.pack(pady=10, fill=tk.BOTH, expand=True)
178
179 self.reloj_label = ttk.Label(self.video_frame, font=("Arial", 14), foreground=" #E0E0E0")
180 self.reloj_label.pack(pady=5)
181
182 ttk.Label(right_frame, text="Configuración", font=("Arial", 16, "bold"), foreground=" #4A90E2").pack(pady=(10, 5))
183
184 config_frame = ttk.Frame(right_frame)
185 config_frame.pack(pady=5)
186 ttk.Label(config_frame, text="🌐 IP Local:").grid(row=0, column=0, sticky='e', pady=5)
187 self.ip_entry = ttk.Entry(config_frame, width=20, style="Custom.TEntry")
188 self.ip_entry.grid(row=0, column=1, pady=5, padx=(5, 0))
189 self.ip_entry.insert(0, obtener_ip_local())
190
191 ttk.Label(config_frame, text="👤 Usuario:").grid(row=1, column=0, sticky='e', pady=5)
192 self.user_display = ttk.Entry(config_frame, width=20, style="Custom.TEntry")
193 self.user_display.insert(0, "Bowser")
194 self.user_display.config(state="readonly")
195 self.user_display.grid(row=1, column=1, pady=5, padx=(5, 0))
196
197 ttk.Separator(right_frame, orient='horizontal').pack(fill='x', pady=10)
198
199 ttk.Label(right_frame, text="Acciones", font=("Arial", 16, "bold"), foreground=" #4A90E2").pack(pady=(5, 5))
200
201 acciones_frame = ttk.Frame(right_frame)
202 acciones_frame.pack(pady=5)
203
204 self.boton_live = ttk.Button(acciones_frame, text="▶ Live", command=self.toggle_live, style="Custom.TButton")
205 self.boton_live.grid(row=0, column=0, padx=5, pady=5, sticky='ew')

```

Lineas 174-208:

Linea 174: titulo.pack(pady=10)

- Empaqueta (pack) la etiqueta del título con 10 px de padding vertical.

Linea 176: self.imagen_label = tk.Label(self.video_frame, borderwidth=2, relief="flat", bg="#2A2A3D")

- Crea un Label de tkinter en video_frame que mostrará la imagen (video o captura).
- borderwidth=2, relief="flat": Define un borde fino y sin relieve.
- bg="#2A2A3D": Color de fondo oscuro para que destaque la imagen.

Linea 177: self.imagen_label.pack(pady=10, fill=tk.BOTH, expand=True)

- Empaqueta el imagen_label con 10 px de espacio vertical.
- fill=tk.BOTH, expand=True: Hace que ocupe todo el espacio disponible del marco.

Linea 179: self.reloj_label = ttk.Label(self.video_frame, font=("Arial", 14), foreground="#E0E0E0")

- Crea una etiqueta (ttk.Label) para mostrar la hora actual.
- Fuente Arial 14 y texto en color gris claro.

Linea 180: self.reloj_label.pack(pady=5)

- Empaqueta reloj_label con 5 px de padding vertical.

Linea 182: ttk.Label(right_frame, text="Configuración", font=("Arial", 16, "bold"), foreground="#4A90E2").pack(pady=(10, 5))

- En el panel derecho, crea y empaqueta un subtítulo "Configuración".
- Fuente Arial 16 negrita, color azul, con 10 px arriba y 5 px abajo.

Linea 184: config_frame = ttk.Frame(right_frame)

- Crea un nuevo Frame dentro de right_frame para agrupar controles de configuración.

Linea 185: config_frame.pack(pady=5)

- Empaqueta el config_frame con 5 px de padding vertical.

Linea 186: ttk.Label(config_frame, text="🌐 IP Local:").grid(row=0, column=0, sticky='e', pady=5)

- Etiqueta "IP Local:" en la fila 0, columna 0, alineada a la derecha (sticky='e').

Linea 187: self.ip_entry = ttk.Entry(config_frame, width=20, style="Custom.TEntry")

- Campo de texto para mostrar la IP local, 20 caracteres de ancho, con estilo personalizado.

Línea 188: `self.ip_entry.grid(row=0, column=1, pady=5, padx=(5, 0))`

- Coloca el campo de IP en la fila 0, columna 1, con 5 px de margen a la izquierda.

Línea 189: `self.ip_entry.insert(0, obtener_ip_local())`

- Inserta en el campo la IP obtenida por la función `obtener_ip_local()`.

Línea 191: `ttk.Label(config_frame, text="👤 Usuario:").grid(row=1, column=0, sticky='e', pady=5)`

- Etiqueta “Usuario:” en fila 1, columna 0, alineada a la derecha.

Línea 192: `self.user_display = ttk.Entry(config_frame, width=20, style="Custom.TEntry")`

- Campo de texto para mostrar el nombre de usuario, con el mismo ancho y estilo.

Línea 193: `self.user_display.insert(0, "Bowser")`

- Inserta el texto fijo “Bowser” en el campo de usuario.

Línea 194: `self.user_display.config(state="readonly")`

- Configura el campo para que sea de solo lectura, impidiendo ediciones.

Línea 195: `self.user_display.grid(row=1, column=1, pady=5, padx=(5, 0))`

- Empaqueta el campo de usuario en fila 1, columna 1, con márgenes similares al anterior.

Línea 197: `ttk.Separator(right_frame, orient='horizontal').pack(fill='x', pady=10)`

- Añade un separador horizontal que cruza todo el ancho del panel derecho, con 10 px de espacio.

Línea 199: `ttk.Label(right_frame, text="Acciones", font=("Arial", 16, "bold"), foreground="#4A90E2").pack(pady=(5, 5))`

- Subtítulo “Acciones” para la sección de botones, estilo similar al de “Configuración”.

Línea 201: `acciones_frame = ttk.Frame(right_frame)`

- Crea un marco para agrupar los botones de acción.

Línea 202: `acciones_frame.pack(pady=5)`

- Empaqueta `acciones_frame` con 5 px de padding vertical.

Línea 204: `self.boton_live = ttk.Button(acciones_frame, text="▶ Live", command=self.toggle_live, style="Custom.TButton")`

- Botón “Live” que llama a `toggle_live` y usa estilo personalizado.

Línea 205: `self.boton_live.grid(row=0, column=0, padx=5, pady=5, sticky='ew')`

- Coloca el botón en fila 0, columna 0, expandiéndose en el ancho (sticky='ew').

Línea 207: `boton_capturar = ttk.Button(acciones_frame, text="📷 Capturar", command=self.capturar_imagen, style="Custom.TButton")`

- Botón “Capturar” que invoca `capturar_imagen` para sacar una foto del vídeo.

Línea 208: `boton_capturar.grid(row=1, column=0, padx=5, pady=5, sticky='ew')`

- Ubica el botón de captura justo debajo de “Live”, con el mismo margen y expansión.

```
210 self.boton_guardar = ttk.Button(acciones_frame, text="💾 Guardar", command=self.guardar_imagen,
211                               style="Custom.TButton", state='disabled')
212 self.boton_guardar.grid(row=2, column=0, padx=5, pady=5, sticky='ew')
213
214 boton_alerta = ttk.Button(acciones_frame, text="🚨 Alerta", command=self.enviar_alerta, style="Custom.TButton")
215 boton_alerta.grid(row=3, column=0, padx=5, pady=5, sticky='ew')
216
217 boton_analizar = ttk.Button(acciones_frame, text="🔍 Analizar", command=self.analizar_imagen, style="Custom.TButton")
218 boton_analizar.grid(row=4, column=0, padx=5, pady=5, sticky='ew')
219
220 ttk.Separator(right_frame, orient='horizontal').pack(fill='x', pady=10)
221
222 ttk.Label(right_frame, text="Registro de Eventos", font=("Arial", 16, "bold"), foreground="#4A90E2").pack(pady=(5, 5))
223
224 self.log_text = tk.Text(right_frame, height=12, state='disabled', bg="#2A2A3D", fg="#E0E0E0", wrap='word',
225                        font=("Arial", 10))
226 self.log_text.pack(padx=5, pady=5, fill='both', expand=True)
227
228 self.estado_label = ttk.Label(self.root, text="Estado: Esperando acción...", font=("Arial", 12, "italic"),
229                              foreground="#F5A623")
230 self.estado_label.pack(pady=5)
231
232 def _bg_color(self):
233     return "#1E1E2F" if self.dark_mode else "#F0F0F0"
234
235 def _fg_color(self):
236     return "#E0E0E0" if self.dark_mode else "#000000"
237
238 def _entry_bg(self):
239     return "#2A2A3D" if self.dark_mode else "#FFFFFF"
240
241 def _button_bg(self):
242     return "#4A90E2" if self.dark_mode else "#007ACC"
243
244 def _txt_bg_color(self):
```

Líneas 210-244:

Línea 210: `self.boton_guardar = ttk.Button(acciones_frame, text="💾 Guardar", command=self.guardar_imagen,`

- Crea el botón “Guardar” que invoca `self.guardar_imagen`.

Línea 211: `style="Custom.TButton", state='disabled')`

- Aplica estilo personalizado y lo inicia deshabilitado (state='disabled') hasta que haya imagen capturada.

Línea 212: `self.boton_guardar.grid(row=2, column=0, padx=5, pady=5, sticky='ew')`

- Ubica el botón en fila 2, columna 0, con márgenes y expansión horizontal.

Línea 214: `boton_alerta = ttk.Button(acciones_frame, text="🚨 Alerta", command=self.enviar_alerta, style="Custom.TButton")`

- Crea el botón “Alerta” que llama a `self.enviar_alerta` al pulsar.

Línea 215: `boton_alerta.grid(row=3, column=0, padx=5, pady=5, sticky='ew')`

- Coloca el botón de alerta en la siguiente fila con el mismo espaciado.

Línea 217: `boton_analizar = ttk.Button(acciones_frame, text="🔍 Analizar", command=self.analizar_imagen, style="Custom.TButton")`

- Botón “Analizar” para ejecutar `self.analizar_imagen` en la imagen capturada.

Línea 218: `boton_analizar.grid(row=4, column=0, padx=5, pady=5, sticky='ew')`

- Ubica el botón de análisis en fila 4 del marco de acciones.

Línea 220: `ttk.Separator(right_frame, orient='horizontal').pack(fill='x', pady=10)`

- Añade un separador horizontal en el panel derecho con 10 px de espacio.

Línea 222: `ttk.Label(right_frame, text="Registro de Eventos", font=("Arial", 16, "bold"), foreground="#4A90E2").pack(pady=(5, 5))`

- Título “Registro de Eventos” para el área de log, con estilo consistente.

Línea 224: `self.log_text = tk.Text(right_frame, height=12, state='disabled', bg="#2A2A3D", fg="#E0E0E0", wrap='word',`

- Crea un widget Text para mostrar los eventos de la aplicación.
- `height=12`: 12 líneas de alto.
- `state='disabled'`: Lectura solamente.
- `bg, fg`: colores de fondo y texto según modo oscuro.
- `wrap='word'`: Ajuste por palabra.

Línea 225: `font=("Arial", 10))`

- Fuente Arial tamaño 10 para el texto de log.

Línea 226: `self.log_text.pack(padx=5, pady=5, fill='both', expand=True)`

- Empaqueta el área de log con padding y permite que crezca.

Línea 228: `self.estado_label = ttk.Label(self.root, text="Estado: Esperando acción...", font=("Arial", 12, "italic"),`

- Etiqueta de estado en la parte inferior de la ventana principal.
- Texto inicial “Estado: Esperando acción...”, fuente Arial 12 cursiva.

Línea 229: `foreground="#F5A623")`

- Texto en color naranja claro (#F5A623) para destacar el estado.

Línea 230: `self.estado_label.pack(pady=5)`

- Empaqueta la etiqueta de estado con 5 px de espacio vertical.

Línea 232: `def _bg_color(self):`

- Método interno que retorna el color de fondo según el modo oscuro.

Línea 233: return "#1E1E2F" if self.dark_mode else "#F0F0F0"

- Si self.dark_mode es True, fondo oscuro; si no, claro.

Línea 235: def _fg_color(self):

- Método que retorna el color de primer plano (texto) según el modo.

Línea 236: return "#E0E0E0" if self.dark_mode else "#000000"

- Texto claro en modo oscuro, negro en modo claro.

Línea 238: def _entry_bg(self):

- Método que retorna el color de fondo para widgets de entrada.

Línea 239: return "#2A2A3D" if self.dark_mode else "#FFFFFF"

- Fondo oscuro o blanco según el modo.

Línea 241: def _button_bg(self):

- Método que retorna el color de fondo para botones.

Línea 242: return "#4A90E2" if self.dark_mode else "#007ACC"

- Botón azul claro en modo oscuro, azul diferente en claro.

Línea 244: def _txt_bg_color(self):

- Método que retorna el color de fondo para el área de texto (log) según el modo.

```
245     return "#2A2A3D" if self.dark_mode else "#FFFFFF"
246
247 def _txt_fg_color(self):
248     return "#E0E0E0" if self.dark_mode else "#000000"
249
250 def _configurar_estilos(self):
251     bg = self._bg_color()
252     fg = self._fg_color()
253     entry_bg = self._entry_bg()
254     button_bg = self._button_bg()
255
256     self.root.configure(bg=bg)
257     self.style.configure("TFrame", background=bg)
258     self.style.configure("TLabel", background=bg, foreground=fg, font=("Arial", 11))
259     self.style.configure("Custom.TEntry", fieldbackground=entry_bg, foreground=fg, font=("Arial", 11))
260     self.style.configure("Custom.TButton", padding=10, relief="flat", background=button_bg, foreground="white",
261                          font=("Arial", 10, "bold"))
262     self.style.map("Custom.TButton",
263                  background=[("active", "#357ABD"), ("!active", button_bg)],
264                  foreground=[("active", "white"), ("!active", "white")])
265
266 def toggle_dark_mode(self):
267     self.dark_mode = self.var_darkmode.get()
268     self._configurar_estilos()
269     self.imagen_label.configure(bg=self._bg_color())
270     self.log_text.configure(bg=self._txt_bg_color(), fg=self._txt_fg_color())
271     self.estado_label.configure(background=self._bg_color(), foreground=self._fg_color())
272
273 def actualizar_reloj(self):
274     try:
275         hora_actual = datetime.now().strftime("%H:%M:%S")
276         self.reloj_label.config(text=f"🕒 {hora_actual}")
277         self.root.after(1000, self.actualizar_reloj)
278     except tk.TclError:
279         pass
280
```

Líneas 245-279:

Línea 245: return "#2A2A3D" if self.dark_mode else "#FFFFFF"

- Retorna el color de fondo para el área de texto según self.dark_mode.

Linea 247: def _txt_fg_color(self):

- Define el método que retorna el color de primer plano (texto) para el área de texto.

Linea 248: return "#E0E0E0" if self.dark_mode else "#000000"

- Si self.dark_mode es True, devuelve gris claro; si no, negro.

Linea 250: def _configurar_estilos(self):

- Método interno que aplica todos los estilos (ttk.Style) basados en los colores calculados.

Linea 251: bg = self._bg_color()

- Guarda en bg el color de fondo general.

Linea 252: fg = self._fg_color()

- Guarda en fg el color de texto general.

Linea 253: entry_bg = self._entry_bg()

- Guarda en entry_bg el color de fondo para campos de entrada.

Linea 254: button_bg = self._button_bg()

- Guarda en button_bg el color de fondo para botones.

Linea 256: self.root.configure(bg=bg)

- Aplica el color de fondo a la ventana principal.

Linea 257: self.style.configure("TFrame", background=bg)

- Configura el estilo de todos los Frame con el fondo bg.

Linea 258: self.style.configure("TLabel", background=bg, foreground=fg, font=("Arial", 11))

- Ajusta etiquetas con fondo bg, texto fg y fuente Arial 11.

Linea 259: self.style.configure("Custom.TEntry", fieldbackground=entry_bg, foreground=fg, font=("Arial", 11))

- Estilo para campos de entrada con fondo entry_bg, texto fg.

Linea 260: self.style.configure("Custom.TButton", padding=10, relief="flat", background=button_bg, foreground="white",

Linea 261: font=("Arial", 10, "bold"))

- Define botones con 10 px de padding, borde plano, fondo button_bg y fuente negrita.

Linea 262: self.style.map("Custom.TButton",

- Inicia el mapeo de estados para el estilo de botón personalizado.

Linea 263: background=[("active", "#357ABD"), ("!active", button_bg)],

- Cambia el color de fondo a #357ABD cuando el botón esté activo.

Linea 264: foreground=[("active", "white"), ("!active", "white")]

- Mantiene el texto blanco tanto activo como inactivo.

Linea 266: def toggle_dark_mode(self):

- Método que alterna entre modo oscuro y claro.

Linea 267: self.dark_mode = self.var_darkmode.get()

- Actualiza self.dark_mode según el valor de la casilla del menú.

Linea 268: self._configurar_estilos()

- Vuelve a aplicar todos los estilos para reflejar el nuevo modo.

Linea 269: self.imagen_label.configure(bg=self._bg_color())

- Ajusta el fondo del label de la imagen según el modo actual.

Linea 270: self.log_text.configure(bg=self._txt_bg_color(), fg=self._txt_fg_color())

- Actualiza los colores de fondo y texto del área de log.

Linea 271: self.estado_label.configure(background=self._bg_color(), foreground=self._fg_color())

- Cambia los colores de la etiqueta de estado.

Linea 273: def actualizar_reloj(self):

- Método que actualiza el reloj cada segundo.

Linea 274: try:

- Bloque try para capturar posibles errores de tkinter si la ventana ya está cerrada.

Linea 275: hora_actual = datetime.now().strftime("%H:%M:%S")

- Obtiene la hora actual en formato HH:MM:SS.

Linea 276: self.reloj_label.config(text=f"🕒 {hora_actual}")

- Actualiza el texto de reloj_label con la hora.

Linea 277: self.root.after(1000, self.actualizar_reloj)

- Programa la siguiente actualización tras 1 000 ms (1 segundo).

Linea 278: except tk.TclError:

- Captura la excepción TclError si el widget ya no existe.

Línea 279: pass

- Ignora el error silenciosamente, deteniendo la actualización.

```

281 def log_evento(self, mensaje):
282     self.log_text.config(state='normal')
283     self.log_text.insert(tk.END, f"{datetime.now().strftime('%H:%M:%S')} - {mensaje}\n")
284     self.log_text.config(state='disabled')
285     self.log_text.see(tk.END)
286
287 def capturar_imagen(self):
288     if self.live_activado and self.cap:
289         ret, frame = self.cap.read()
290         if ret:
291             timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
292             nombre_archivo = f"{timestamp}.png"
293             carpeta = "capturas"
294             ruta = os.path.join(carpeta, nombre_archivo)
295
296             cv2.imwrite(ruta, frame)
297             self.imagen_capturada_path = ruta
298
299             self.log_evento("📸 Imagen capturada correctamente.")
300             self.estado_label.config(text="✅ Imagen capturada correctamente.", foreground="green")
301
302             img = Image.open(self.imagen_capturada_path)
303             img = img.resize((800, 500))
304             img_tk = ImageTk.PhotoImage(img)
305             self.imagen_label.config(image=img_tk)
306             self.imagen_label.image = img_tk
307
308             self.boton_guardar.config(state='normal')
309         else:
310             self.log_evento("❌ Error capturando imagen.")
311             self.estado_label.config(text="❌ Error capturando imagen.", foreground="red")
312     else:
313         messagebox.showinfo("Info", "No hay video en vivo activo para capturar.")
314
315 def guardar_imagen(self):
316     ruta_origen = self.imagen_capturada_path

```

Líneas 281-316:

Línea 281: def log_evento(self, mensaje):

- Define el método log_evento que añade entradas al widget de texto de registro.

Línea 282: self.log_text.config(state='normal')

- Cambia el estado del Text de “disabled” a “normal” para permitir insertar texto.

Línea 283: self.log_text.insert(tk.END, f"{datetime.now().strftime('%H:%M:%S')} - {mensaje}\n")

- Inserta en la última línea (tk.END) la hora actual y el mensaje recibido, seguido de salto de línea.

Línea 284: self.log_text.config(state='disabled')

- Vuelve a poner el widget en modo “disabled” para prevenir edición manual.

Línea 285: self.log_text.see(tk.END)

- Desplaza la vista hasta el final del texto, mostrando la última entrada agregada.

Línea 287: def capturar_imagen(self):

- Define el método que captura un fotograma del vídeo en vivo cuando está activo.

Línea 288: if self.live_activado and self.cap:

- Comprueba que el vídeo en vivo esté activado y que el objeto VideoCapture exista.

Linea 289: ret, frame = self.cap.read()

- Lee un fotograma de la cámara; ret indica éxito y frame contiene la imagen.

Linea 290: if ret:

- Si la lectura fue exitosa, ejecuta el bloque para guardar y mostrar la imagen.

Linea 291: timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")

- Genera una cadena con fecha y hora para nombrar el archivo: YYYYMMDD_HHMMSS.

Linea 292: nombre_archivo = f"{timestamp}.png"

- Crea el nombre del archivo añadiendo la extensión .png.

Linea 293: carpeta = "capturas"

- Define la carpeta donde se almacenarán las capturas.

Linea 294: ruta = os.path.join(carpeta, nombre_archivo)

- Construye la ruta completa (carpeta/nombre_archivo) de forma portable.

Linea 296: cv2.imwrite(ruta, frame)

- Guarda la imagen (frame) en disco en la ruta especificada en formato PNG.

Linea 297: self.imagen_capturada_path = ruta

- Almacena la ruta de la imagen capturada en un atributo para uso posterior.

Linea 299: self.log_evento("📷 Imagen capturada correctamente.")

- Registra en el log un mensaje indicando que la captura fue exitosa.

Linea 300: self.estado_label.config(text="✅ Imagen capturada correctamente.", foreground="green")

- Actualiza la etiqueta de estado con un mensaje de éxito en color verde.

Linea 302: img = Image.open(self.imagen_capturada_path)

- Abre la imagen usando Pillow para poder manipularla.

Linea 303: img = img.resize((800, 500))

- Redimensiona la imagen a 800×500 píxeles para que encaje en el Label.

Linea 304: img_tk = ImageTk.PhotoImage(img)

- Convierte la imagen a un objeto compatible con tkinter.

Linea 305: self.imagen_label.config(image=img_tk)

- Asigna la imagen convertida al Label de la interfaz.

Línea 306: `self.imagen_label.image = img_tk`

- Guarda referencia a `img_tk` en el widget para evitar que el recolector de basura la destruya.

Línea 308: `self.boton_guardar.config(state='normal')`

- Habilita el botón “Guardar” ahora que existe una imagen capturada.

Línea 309: `else:`

- Si `ret` es `False`, la lectura del fotograma falló, se ejecuta el bloque de error.

Línea 310: `self.log_evento("❌ Error capturando imagen.")`

- Registra en el log el fallo de captura.

Línea 311: `self.estado_label.config(text="❌ Error capturando imagen.", foreground="red")`

- Muestra mensaje de error en rojo en la etiqueta de estado.

Línea 312: `else:`

- Si no hay vídeo en vivo activo o `self.cap` es `None`, se muestra información.

Línea 313: `messagebox.showinfo("Info", "No hay video en vivo activo para capturar.")`

- Muestra un cuadro de información indicando que no puede capturar sin vídeo.

Línea 314: *(línea en blanco)*

Línea 315: `def guardar_imagen(self):`

- Define el método que copia la imagen capturada a una ubicación elegida por el usuario.

Línea 316: `ruta_origen = self.imagen_capturada_path`

- Asigna a `ruta_origen` la ruta almacenada de la última captura para copiarla.

```

317 ruta_destino = filedialog.asksaveasfilename(defaultextension=".png", filetypes=[("PNG files", "*.png")])
318 if ruta_destino:
319     try:
320         shutil.copy(ruta_origen, ruta_destino)
321         self.log_evento(f"📁 Imagen guardada en {ruta_destino}")
322         self.estado_label.config(text=f"📁 Imagen guardada con éxito.", foreground="green")
323     except Exception as e:
324         messagebox.showerror("Error", f"No se pudo guardar la imagen: {e}")
325         self.log_evento(f"❌ Error guardando imagen: {e}")
326         self.estado_label.config(text="❌ Error guardando imagen.", foreground="red")
327
328 def enviar_email_alerta(self, asunto: str, cuerpo: str):
329     """
330     Envía un correo de alerta vía SMTP usando TLS (smtp.gmail.com:587).
331     Los destinatarios fijos: angel.mejia.e@uni.pe.
332     """
333     email_sender = os.getenv("EMAIL_SENDER")
334     email_password = os.getenv("EMAIL_PASSWORD")
335     destinatarios = [
336         "angel.mejia.e@uni.pe"
337     ]
338
339     if not email_sender or not email_password:
340         self.log_evento("❌ Error: no se encontraron credenciales de email en variables de entorno.")
341         return
342
343     mensaje = MIMEText(cuerpo, "plain", _charset="utf-8")
344     mensaje["From"] = email_sender
345     mensaje["To"] = ", ".join(destinatarios)
346     mensaje["Subject"] = asunto
347     mensaje.attach(MIMEText(cuerpo, "plain", _charset="utf-8"))
348
349     try:
350         context = ssl.create_default_context()
351         with smtplib.SMTP("smtp.gmail.com", 587) as server:

```

Lineas **317-351**:

Linea 317: `ruta_destino = filedialog.asksaveasfilename(defaultextension=".png", filetypes=[("PNG files", "*.png")])`

- Abre un diálogo para que el usuario elija dónde guardar la imagen.
- `defaultextension`: sugiere “.png”; `filetypes`: filtra a archivos PNG.

Linea 318: `if ruta_destino:`

- Comprueba si el usuario seleccionó una ruta (no canceló el diálogo).

Linea 319: `try:`

- Inicia un bloque para intentar copiar el archivo y capturar posibles errores.

Linea 320: `shutil.copy(ruta_origen, ruta_destino)`

- Copia el archivo desde la ruta original (`ruta_origen`) a la ruta destino.

Linea 321: `self.log_evento(f"💾 Imagen guardada en {ruta_destino}")`

- Registra en el log el éxito indicando la ruta donde se guardó.

Linea 322: `self.estado_label.config(text="💾 Imagen guardada con éxito.", foreground="green")`

- Muestra en la interfaz un mensaje de éxito en color verde.

Linea 323: `except Exception as e:`

- Captura cualquier excepción (`e`) que ocurra durante la copia.

Linea 324: `messagebox.showerror("Error", f"No se pudo guardar la imagen: {e}")`

- Muestra un cuadro de error con el mensaje de la excepción.

Linea 325: `self.log_evento(f"❌ Error guardando imagen: {e}")`

- Registra en el log el detalle del error.

Linea 326: `self.estado_label.config(text="❌ Error guardando imagen.", foreground="red")`

- Actualiza la etiqueta de estado con un mensaje de fallo en rojo.

Linea 328: `def enviar_email_alerta(self, asunto: str, cuerpo: str):`

- Define el método que envía un correo de alerta usando SMTP.
- Parámetros: asunto (subject) y cuerpo (body) del mensaje.

Linea 329: `"""`

- Inicio de la cadena de documentación (docstring) del método.

Linea 330: Envía un correo de alerta vía SMTP usando TLS (smtp.gmail.com:587).

- Describe que emplea TLS por el puerto 587 de Gmail.

Linea 331: Los destinatarios fijos: angel.mejia.e@uni.pe.

- Indica que el correo siempre se envía a esa dirección fija.

Linea 332: ""

- Fin de la docstring.

Linea 333: email_sender = os.getenv("EMAIL_SENDER")

- Obtiene de las variables de entorno el correo remitente.

Linea 334: email_password = os.getenv("EMAIL_PASSWORD")

- Obtiene de las variables de entorno la contraseña o app-password.

Linea 335: destinatarios = [

- Inicia una lista de destinatarios a quienes se enviará el correo.

Linea 336: angel.mejia.e@uni.pe

- Dirección fija incluida en la lista.

Linea 337:]

- Cierre de la lista de destinatarios.

Linea 339: if not email_sender or not email_password:

- Verifica que existan ambas credenciales en las variables de entorno.

Linea 340: self.log_evento(" ❌ Error: no se encontraron credenciales de email en variables de entorno.")

- Registra en el log la falta de credenciales.

Linea 341: return

- Sale del método si no están definidas las credenciales.

Linea 343: mensaje = MIMEMultipart()

- Crea el objeto multipart para componer el correo.

Linea 344: mensaje["From"] = email_sender

- Establece el campo "From" con el correo remitente.

Linea 345: mensaje["To"] = ", ".join(destinatarios)

- Establece el campo "To" con la(s) dirección(es) de destinatario(s).

Línea 346: mensaje["Subject"] = asunto

- Asigna el asunto al correo.

Línea 347: mensaje.attach(MIMEText(cuerpo, "plain", _charset="utf-8"))

- Adjunta el cuerpo como texto plano UTF-8 al mensaje.

Línea 349: try:

- Inicia el bloque para la conexión SMTP y envío del correo.

Línea 350: context = ssl.create_default_context()

- Crea un contexto SSL/TLS predeterminado para cifrar la conexión.

Línea 351: with smtplib.SMTP("smtp.gmail.com", 587) as server:

- Abre la conexión con el servidor SMTP de Gmail en el puerto 587, gestionado automáticamente por el gestor de contexto.

```
352         server.ehlo()
353         server.starttls(context=context)
354         server.ehlo()
355         server.login(email_sender, email_password)
356         server.sendmail(
357             from_addr=email_sender,
358             to_addrs=destinatarios,
359             msg=mensaje.as_string().encode("utf-8")
360         )
361         self.log_evento(f"✉ Email de alerta enviado a: {' '.join(destinatarios)}")
362     except Exception as e:
363         self.log_evento(f"✖ Error enviando email: {e}")
364
365     def enviar_alerta(self):
366         # Mostrar mensaje de alerta emergente
367         if self.ultimos_objetos_detectados:
368             objetos_texto = " ".join(self.ultimos_objetos_detectados)
369             mensaje_alerta = f"⚠ Alerta: Amenaza detectada. Objetos: {objetos_texto}"
370         else:
371             mensaje_alerta = "⚠ Alerta: Amenaza detectada. Sin objetos identificados."
372         messagebox.showwarning("⚠ Alerta de Seguridad", mensaje_alerta)
373
374         # Reproducir sonido de alerta
375         try:
376             winsound.Beep(1000, 500) # Frecuencia: 1000 Hz, Duración: 500 ms
377         except Exception as e:
378             self.log_evento(f"✖ Error reproduciendo sonido: {e}")
379
380         self.log_evento(f"📧 Alerta enviada")
381         self.estado_label.config(text=f"📧 Alerta de seguridad enviada.", foreground="orange")
382
383         # Enviar SMS con Twilio a multiples destinatarios
384         try:
385             mensaje_sms = mensaje_alerta.replace("\n", " ") # Asegurar que el mensaje esté en una sola línea
386             for destinatario in self.destinatarios.phone_numbers:
```

Líneas 352-386:

Línea 352: server.ehlo()

- Envía el comando EHLO al servidor SMTP para identificarse y obtener capacidades extendidas.

Línea 353: server.starttls(context=context)

- Inicia el cifrado TLS de la conexión usando el contexto SSL creado previamente.

Línea 354: server.ehlo()

- Vuelve a enviar EHLO tras activar TLS, para renegociar capacidades seguras.

Linea 355: server.login(email_sender, email_password)

- Autentica en el servidor SMTP con el correo remitente y su contraseña.

Linea 356: server.sendmail(

- Comienza la llamada para enviar el correo.

Linea 357: from_addr=email_sender,

- Parametra la dirección de origen del mensaje.

Linea 358: to_addrs=destinatarios,

- Lista de destinatarios que recibirán el correo.

Linea 359: msg=msg.as_string().encode("utf-8")

- Convierte el objeto MIMEMultipart a cadena y luego a bytes UTF-8 para transmisión.

Linea 360:)

- Cierra la llamada a sendmail.

Linea 361: self.log_evento(f"✉ Email de alerta enviado a: {' '.join(destinatarios)}")

- Registra en el log que el correo fue enviado exitosamente y a qué direcciones.

Linea 362: except Exception as e:

- Captura cualquier excepción que ocurra durante la conexión o envío de email.

Linea 363: self.log_evento(f"✖ Error enviando email: {e}")

- Registra en el log el error ocurrido al tratar de enviar el correo.

Linea 365: def enviar_alerta(self):

- Define el método que genera y envía las alertas (ventana, sonido, SMS y email).

Linea 366: # Mostrar mensaje de alerta emergente

- Comentario que indica que a continuación se mostrará un diálogo emergente.

Linea 367: if self.ultimos_objetos_detectados:

- Comprueba si hay objetos detectados para incluirlos en el mensaje de alerta.

Linea 368: objetos_texto = ", ".join(self.ultimos_objetos_detectados)

- Une los nombres de los objetos detectados en una cadena separada por comas.

Linea 369: mensaje_alerta = f"⚠ Alerta: Amenaza detectada. Objetos: {objetos_texto}"

- Construye el texto de alerta incluyendo la lista de objetos.

Linea 370: else:

- Si no hubo detecciones, usa un mensaje genérico.

Linea 371: mensaje_alerta = "⚠️ Alerta: Amenaza detectada. Sin objetos identificados."

- Mensaje indicando que no se reconoció ningún objeto.

Linea 372: messagebox.showwarning("⚠️ Alerta de Seguridad", mensaje_alerta)

- Muestra un cuadro de advertencia con el título y el mensaje de alerta.

Linea 374: # Reproducir sonido de alerta

- Comentario que introduce la sección de reproducción de audio.

Linea 375: try:

- Inicia un bloque try para intentar emitir un sonido.

Linea 376: winsound.Beep(1000, 500) # Frecuencia: 1000 Hz, Duración: 500 ms

- Emite un pitido de 1000 Hz durante 500 ms en Windows.

Linea 377: except Exception as e:

- Captura errores si no se puede reproducir sonido (p.ej., en otro SO).

Linea 378: self.log_evento(f"❌ Error reproduciendo sonido: {e}")

- Registra en el log el error de audio.

Linea 380: self.log_evento("🔔 Alerta enviada")

- Registra en el log que se inició el proceso de alerta.

Linea 381: self.estado_label.config(text="🔔 Alerta de seguridad enviada.", foreground="orange")

- Actualiza la etiqueta de estado indicando éxito de alerta en color naranja.

Linea 383: # Enviar SMS con Twilio a múltiples destinatarios

- Comentario que introduce la sección de envío de SMS.

Linea 384: try:

- Inicia un bloque try para el envío de mensajes de texto.

Linea 385: mensaje_sms = mensaje_alerta.replace("\n", " ") # Asegurar que el mensaje esté en una sola línea

- Sustituye saltos de línea por espacios para envío en una sola línea de SMS.

Linea 386: for destinatario in self.destinatarios_phone_numbers:

- Comienza un bucle para iterar sobre cada número de teléfono destinatario.

```
387 # Validación básica del formato del número
388 if not destinatario.startswith("+") or len(destinatario) < 10:
389     self.log_evento(f"❌ Número inválido: {destinatario}")
390     continue
391 mensaje = self.twilio_client.messages.create(
392     body=mensaje_sms,
393     from_=self.twilio_phone_number,
394     to=destinatario
395 )
396 self.log_evento(f"✅ SMS enviado al número {destinatario}. SID: {mensaje.sid}")
397 self.estado_label.config(text="✅ SMS enviado a todos los números con éxito.", foreground="green")
398 except Exception as e:
399     self.log_evento(f"❌ Error enviando SMS: {str(e)}")
400     self.estado_label.config(text="❌ Error enviando SMS.", foreground="red")
401
402 # Enviar correo electrónico adicional (nueva parte)
403 try:
404     asunto_email = "Alerta de Seguridad: Objeto Detectado"
405     cuerpo_email = mensaje_alerta + "\n\n"
406     cuerpo_email += f"Fecha y hora: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}"
407     self.enviar_email_alerta(asunto_email, cuerpo_email)
408 except Exception:
409     pass
410
411 def mostrar_video(self):
412     while self.live activado and self.cap:
413         ret, frame = self.cap.read()
414         if not ret:
415             self.log_evento("❌ No se pudo leer frame de la cámara")
416             break
417         results = self.model(frame)
418         annotated_frame = results[0].plot()
419         annotated_frame = cv2.cvtColor(annotated_frame, cv2.COLOR_BGR2RGB)
420         annotated_frame = cv2.resize(annotated_frame, (800, 500))
421         img = Image.fromarray(annotated_frame)
```

Líneas 387-421:

Línea 387: # Validación básica del formato del número

- Comentario que señala que se hará una validación de los números de teléfono antes de enviar SMS.

Línea 388: if not destinatario.startswith("+") or len(destinatario) < 10:

- Verifica que el número comience con "+" y tenga al menos 10 caracteres.

Línea 389: self.log_evento(f"❌ Número inválido: {destinatario}")

- Registra en el log si el número no es válido.

Línea 390: continue

- Salta al siguiente número si el actual no es válido.

Línea 391: mensaje = self.twilio_client.messages.create(

- Llama a la API de Twilio para crear y enviar un mensaje SMS.

Línea 392: body=mensaje_sms,

- Contenido del mensaje que se enviará.

Línea 393: from_=self.twilio_phone_number,

- Número registrado en Twilio que se usará como remitente.

Línea 394: to=destinatario

- Número de teléfono del destinatario.

Línea 395:)

- Fin de la llamada a create.

Línea 396: self.log_evento(f"✅ SMS enviado al número {destinatario}. SID: {mensaje.sid}")

- Registra en el log que el SMS fue enviado correctamente, incluyendo su SID.

Línea 397: `self.estado_label.config(text=" 📱 SMS enviado a todos los números con éxito.", foreground="green")`

- Actualiza la etiqueta de estado indicando que los SMS fueron enviados exitosamente.

Línea 398: `except Exception as e:`

- Captura cualquier excepción que ocurra durante el envío de SMS.

Línea 399: `self.log_evento(f" ❌ Error enviando SMS: {str(e)}")`

- Registra en el log el mensaje de error.

Línea 400: `self.estado_label.config(text=" ❌ Error enviando SMS.", foreground="red")`

- Actualiza la etiqueta de estado indicando que hubo un error en el envío de SMS.

Línea 402: `# Enviar correo electrónico adicional (nueva parte)`

- Comentario que indica que se va a enviar un correo como parte de la alerta.

Línea 403: `try:`

- Inicia un bloque try para enviar el email adicional.

Línea 404: `asunto_email = "Alerta de Seguridad: Objeto Detectado"`

- Define el asunto del correo electrónico de alerta.

Línea 405: `cuerpo_email = mensaje_alerta + "\n\n"`

- Crea el cuerpo del email comenzando con el mensaje de alerta.

Línea 406: `cuerpo_email += f"Fecha y hora: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}"`

- Añade al cuerpo la fecha y hora actuales.

Línea 407: `self.enviar_email_alerta(asunto_email, cuerpo_email)`

- Llama a la función que envía el correo con el asunto y el cuerpo definidos.

Línea 408: `except Exception:`

- Captura cualquier error ocurrido durante el intento de enviar el email.

Línea 409: `pass`

- Ignora el error (no se realiza acción si falla el envío).

Línea 411: `def mostrar_video(self):`

- Define la función que transmite video en tiempo real y realiza detección de objetos.

Línea 412: while self.live_activado and self.cap:

- Mientras la transmisión esté activada y la cámara abierta, continúa ejecutando.

Línea 413: ret, frame = self.cap.read()

- Captura un nuevo fotograma (frame) desde la cámara.

Línea 414: if not ret:

- Verifica si hubo un error al capturar el fotograma.

Línea 415: self.log_evento("✗ No se pudo leer frame de la cámara")

- Registra en el log si falló la captura del fotograma.

Línea 416: break

- Sale del bucle si no se pudo capturar un fotograma.

Línea 417: results = self.model(frame)

- Pasa el fotograma por el modelo YOLO para detectar objetos.

Línea 418: annotated_frame = results[0].plot()

- Obtiene el fotograma anotado con los resultados de la detección (cajas, etiquetas).

Línea 419: annotated_frame = cv2.cvtColor(annotated_frame, cv2.COLOR_BGR2RGB)

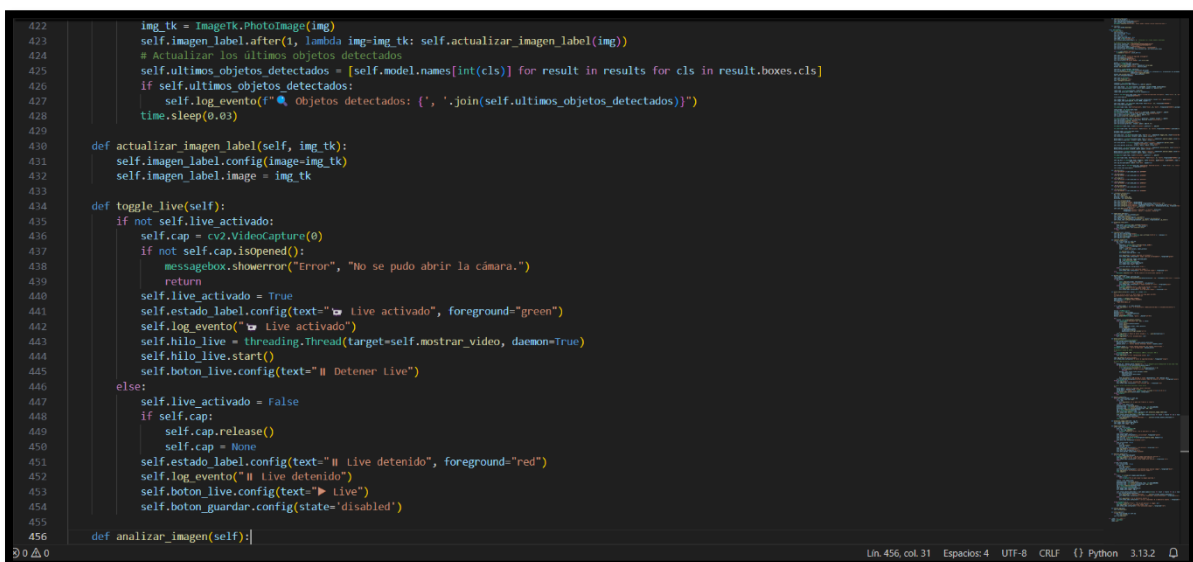
- Convierte el formato de color de BGR (OpenCV) a RGB (PIL/Tkinter).

Línea 420: annotated_frame = cv2.resize(annotated_frame, (800, 500))

- Redimensiona el fotograma a 800x500 píxeles.

Línea 421: img = Image.fromarray(annotated_frame)

- Convierte el arreglo de NumPy (fotograma) a objeto Image de PIL.



```
422     img_tk = ImageTk.PhotoImage(img)
423     self.imagen_label.after(1, lambda img=img_tk: self.actualizar_imagen_label(img))
424     # Actualizar los últimos objetos detectados
425     self.ultimos_objetos_detectados = [self.model.names[int(cls)] for result in results for cls in result boxes.cls]
426     if self.ultimos_objetos_detectados:
427         self.log_evento(f"🔍 Objetos detectados: {' '.join(self.ultimos_objetos_detectados)}")
428     time.sleep(0.03)
429
430 def actualizar_imagen_label(self, img_tk):
431     self.imagen_label.config(image=img_tk)
432     self.imagen_label.image = img_tk
433
434 def toggle_live(self):
435     if not self.live_activado:
436         self.cap = cv2.VideoCapture(0)
437         if not self.cap.isOpened():
438             messagebox.showerror("Error", "No se pudo abrir la cámara.")
439             return
440         self.live_activado = True
441         self.estado_label.config(text="🟢 Live activado", foreground="green")
442         self.log_evento("🟢 Live activado")
443         self.hilo_live = threading.Thread(target=self.mostrar_video, daemon=True)
444         self.hilo_live.start()
445         self.boton_live.config(text="⏹ Detener Live")
446     else:
447         self.live_activado = False
448         if self.cap:
449             self.cap.release()
450             self.cap = None
451         self.estado_label.config(text="🔴 Live detenido", foreground="red")
452         self.log_evento("🔴 Live detenido")
453         self.boton_live.config(text="▶ Live")
454         self.boton_guardar.config(state='disabled')
455
456 def analizar_imagen(self):
```

(PhotoImage).

Línea 423: self.imagen_label.after(1, lambda img=imgTk: self.actualizar_imagen_label(img))

- Programa una llamada asíncrona para actualizar la etiqueta de imagen (Label) con la imagen capturada.

Línea 424: # Actualizar los últimos objetos detectados

- Comentario que indica que se va a actualizar la lista de objetos reconocidos.

Línea 425: self.ultimos_objetos_detectados = [self.model.names[int(cls)] for result in results for cls in result['boxes'].cls]

- Extrae las clases detectadas del modelo YOLO y las traduce a nombres legibles.

Línea 426: if self.ultimos_objetos_detectados:

- Comprueba si se detectó al menos un objeto en el fotograma.

Línea 427: self.log_evento(f"🔍 Objetos detectados: {', '.join(self.ultimos_objetos_detectados)}")

- Registra los nombres de los objetos detectados en el log.

Línea 428: time.sleep(0.03)

- Pausa brevemente la ejecución para no sobrecargar el hilo de procesamiento (30 ms por fotograma).

Línea 430: def actualizar_imagen_label(self, imgTk):

- Define el método que actualiza visualmente la etiqueta de imagen con una nueva imagen.

Línea 431: self.imagen_label.config(image=imgTk)

- Asigna la imagen a la propiedad image del Label.

Línea 432: self.imagen_label.image = imgTk

- Se guarda una referencia para evitar que la imagen sea recolectada por el recolector de basura.

Línea 434: def toggle_live(self):

- Define el método que alterna el estado del video en vivo (iniciar/detener).

Línea 435: if not self.live_activado:

- Si el video en vivo está desactivado, se procederá a activarlo.

Línea 436: self.cap = cv2.VideoCapture(0)

- Se crea un objeto de captura de video para usar la cámara por defecto (0).

Línea 437: if not self.cap.isOpened():

- Verifica si la cámara se pudo abrir correctamente.

Línea 438: `messagebox.showerror("Error", "No se pudo abrir la cámara.")`

- Muestra un mensaje de error si la cámara no está disponible.

Línea 439: `return`

- Sale del método si la cámara no se abrió.

Línea 440: `self.live_activado = True`

- Cambia el estado de `live_activado` a verdadero.

Línea 441: `self.estado_label.config(text="📺 Live activado", foreground="green")`

- Actualiza el texto del estado para indicar que el live está activo.

Línea 442: `self.log_evento("📺 Live activado")`

- Agrega una entrada al log registrando que el video en vivo fue activado.

Línea 443: `self.hilo_live = threading.Thread(target=self.mostrar_video, daemon=True)`

- Crea un hilo nuevo que ejecutará la función `mostrar_video`.

Línea 444: `self.hilo_live.start()`

- Inicia el hilo de video en vivo.

Línea 445: `self.boton_live.config(text="⏏ Detener Live")`

- Cambia el texto del botón para reflejar que ahora se puede detener el live.

Línea 446: `else:`

- Si ya estaba activado, se procederá a desactivarlo.

Línea 447: `self.live_activado = False`

- Cambia el estado a falso para detener el live.

Línea 448: `if self.cap:`

- Verifica que exista un objeto de captura.

Línea 449: `self.cap.release()`

- Libera la cámara.

Línea 450: `self.cap = None`

- Elimina la referencia al objeto de cámara.

Línea 451: `self.estado_label.config(text="⏏ Live detenido", foreground="red")`

- Actualiza el estado para indicar que el live ha sido detenido.

Línea 452: `self.log_evento(" || Live detenido")`

- Agrega una entrada en el log indicando que se detuvo el video en vivo.

Línea 453: `self.boton_live.config(text="▶ Live")`

- Cambia el texto del botón para mostrar que se puede volver a activar.

Línea 454: `self.boton_guardar.config(state='disabled')`

- Desactiva el botón de guardar imagen cuando el live está apagado.

Línea 456: `def analizar_imagen(self):`

- Define el método que analiza una imagen previamente capturada con YOLO.

```
C:\Users\ASUS > OneDrive > Escritorio > Seguridad (renovado).py > SeguridadApp > analizar_imagen
121 class SeguridadApp:
456 def analizar_imagen(self):
457     if not self.imagen_capturada_path:
458         messagebox.showinfo("Info", "No hay imagen capturada para analizar.")
459         self.log_evento("✗ No hay imagen capturada para analizar.")
460         self.estado_label.config(text="✗ No hay imagen para analizar.", foreground="red")
461         return
462
463     if self.live_activado:
464         self.live_activado = False
465         if self.cap:
466             self.cap.release()
467             self.cap = None
468         self.estado_label.config(text="|| Live detenido para analizar imagen.", foreground="blue")
469         self.log_evento("|| Live detenido para analizar imagen.")
470         time.sleep(0.1)
471
472     try:
473         frame = cv2.imread(self.imagen_capturada_path)
474         if frame is None:
475             raise Exception("No se pudo cargar la imagen capturada.")
476
477         results = self.model(frame)
478         annotated_frame = results[0].plot()
479         annotated_frame = cv2.cvtColor(annotated_frame, cv2.COLOR_BGR2RGB)
480         annotated_frame = cv2.resize(annotated_frame, (800, 500))
481         img = Image.fromarray(annotated_frame)
482         img_tk = ImageTk.PhotoImage(img)
483         self.imagen_label.config(image=img_tk)
484         self.imagen_label.image = img_tk
485
486         self.ultimos_objetos_detectados = [self.model.names[int(cls)] for result in results for cls in result boxes.cls]
487         if self.ultimos_objetos_detectados:
488             self.log_evento(f"🔍 Objetos detectados: {', '.join(self.ultimos_objetos_detectados)}")
489             self.estado_label.config(text="🔍 Análisis completado: Objetos detectados.", foreground="blue")
490         else:
491             self.log_evento("🔍 No se detectaron objetos.")
```

Líneas 457-491:

Línea 457: `if not self.imagen_capturada_path:`

- Verifica si existe una imagen previamente capturada. Si no hay, se detiene el análisis.

Línea 458: `messagebox.showinfo("Info", "No hay imagen capturada para analizar.")`

- Muestra una ventana informativa indicando que no hay imagen disponible.

Línea 459: `self.log_evento("✗ No hay imagen capturada para analizar.")`

- Registra el evento en el log, marcándolo como un error.

Línea 460: `self.estado_label.config(text="✗ No hay imagen para analizar.", foreground="red")`

- Cambia la etiqueta de estado para indicar que no hay imagen que analizar.

Línea 461: return

- Termina la ejecución del método si no hay imagen capturada.

Línea 463: if self.live_activado:

- Si el video en vivo está activado, se detiene para evitar conflicto durante el análisis.

Línea 464: self.live_activado = False

- Cambia el estado a False para detener el live.

Línea 465: if self.cap:

- Verifica que el objeto de captura de cámara exista.

Línea 466: self.cap.release()

- Libera los recursos de la cámara.

Línea 467: self.cap = None

- Elimina la referencia al objeto de captura.

Línea 468: self.estado_label.config(text="|| Live detenido para analizar imagen.", foreground="blue")

- Actualiza el texto de estado indicando que el live fue detenido para análisis.

Línea 469: self.log_evento("|| Live detenido para analizar imagen.")

- Agrega el evento al log.

Línea 470: time.sleep(0.1)

- Espera una fracción de segundo para asegurar la liberación de la cámara.

Línea 472: try:

- Inicia un bloque try para capturar posibles errores durante el análisis.

Línea 473: frame = cv2.imread(self.imagen_capturada_path)

- Carga la imagen capturada desde el disco usando OpenCV.

Línea 474: if frame is None:

- Verifica si la imagen no se cargó correctamente.

Línea 475: raise Exception("No se pudo cargar la imagen capturada.")

- Lanza una excepción si la imagen no existe o no pudo leerse.

Línea 477: results = self.model(frame)

- Ejecuta el modelo YOLO sobre la imagen cargada para obtener predicciones.

Línea 478: annotated_frame = results[0].plot()

- Dibuja las cajas y etiquetas sobre la imagen con los resultados de detección.

Línea 479: annotated_frame = cv2.cvtColor(annotated_frame, cv2.COLOR_BGR2RGB)

- Convierte la imagen de BGR (formato OpenCV) a RGB para su visualización en Tkinter.

Línea 480: annotated_frame = cv2.resize(annotated_frame, (800, 500))

- Cambia el tamaño de la imagen anotada para adaptarla al espacio de la interfaz.

Línea 481: img = Image.fromarray(annotated_frame)

- Convierte la imagen en un objeto Image de PIL para poder manipularla visualmente.

Línea 482: img_tk = ImageTk.PhotoImage(img)

- Convierte la imagen PIL en un formato compatible con Tkinter (PhotoImage).

Línea 483: self.imagen_label.config(image=img_tk)

- Establece la imagen procesada como contenido del Label.

Línea 484: self.imagen_label.image = img_tk

- Guarda una referencia a la imagen para evitar que sea recolectada por el recolector de basura.

Línea 486: self.ultimos_objetos_detectados = [self.model.names[int(cls)] for result in results for cls in result.bboxes.cls]

- Extrae los nombres de los objetos detectados y los guarda en la lista correspondiente.

Línea 487: if self.ultimos_objetos_detectados:

- Verifica si se han detectado objetos en la imagen.

Línea 488: self.log_evento(f"🔍 Objetos detectados: {', '.join(self.ultimos_objetos_detectados)}")

- Registra los objetos encontrados en el log.

Línea 489: self.estado_label.config(text="🔍 Análisis completado: Objetos detectados.", foreground="blue")

- Actualiza la etiqueta de estado indicando éxito en el análisis con detección.

Línea 490: else:

- Si no se detectaron objetos, se ejecuta la alternativa.

Línea 491: self.log_evento("🔍 No se detectaron objetos.")

- Registra en el log que no hubo detección de objetos.

```
C:\Users\ASUS> OneDrive > Escritorio > Seguridad (renovado).py > SeguridadApp > analizar_imagen
121 class SeguridadApp:
456     def analizar_imagen(self):
491         self.log_evento("🔍 No se detectaron objetos.")
492         self.estado_label.config(text="🔍 Análisis completado: No se detectaron objetos.", foreground="blue")
493
494     except Exception as e:
495         messagebox.showerror("Error", f"No se pudo analizar la imagen: {e}")
496         self.log_evento(f"❌ Error analizando imagen: {e}")
497         self.estado_label.config(text="❌ Error analizando imagen.", foreground="red")
498
499     def iniciar_app(self):
500         self.root.mainloop()
501
502     def cerrar_app(self):
503         if self.live_activated and self.cap:
504             self.cap.release()
505             self.root.destroy()
506
507 if __name__ == "__main__":
508     login = LoginApp()
509     login.run()
510
```

Líneas 492-509:

Línea 492: `self.estado_label.config(text="🔍 Análisis completado: No se detectaron objetos.", foreground="blue")`

- Actualiza el estado en pantalla para informar que el análisis concluyó sin encontrar objetos.

Línea 494: `except Exception as e:`

- Captura cualquier error que ocurra durante el análisis de la imagen.

Línea 495: `messagebox.showerror("Error", f"No se pudo analizar la imagen: {e}")`

- Muestra una ventana emergente indicando el error encontrado durante el análisis.

Línea 496: `self.log_evento(f"❌ Error analizando imagen: {e}")`

- Registra el error en el historial de eventos de la aplicación.

Línea 497: `self.estado_label.config(text="❌ Error analizando imagen.", foreground="red")`

- Actualiza el estado visible en la GUI para reflejar que ocurrió un fallo en el análisis.

Línea 499: `def iniciar_app(self):`

- Define el método para iniciar la interfaz principal del sistema de seguridad.

Línea 500: `self.root.mainloop()`

- Inicia el bucle principal de eventos de la ventana Tkinter (self.root).

Línea 502: def cerrar_app(self):

- Define el método para cerrar la aplicación de forma segura.

Línea 503: if self.live_activado and self.cap:

- Verifica si el modo en vivo está activo y hay una cámara abierta.

Línea 504: self.cap.release()

- Libera la cámara si está en uso, evitando que quede bloqueada.

Línea 505: self.root.destroy()

- Cierra completamente la ventana principal del sistema de seguridad.

Línea 507: if __name__ == "__main__":

- Verifica si el script se está ejecutando directamente (y no como módulo importado).

Línea 508: login = LoginApp()

- Crea una instancia de la clase LoginApp, que muestra la interfaz de inicio de sesión.

Línea 509: login.run()

- Ejecuta el método run() para lanzar el ciclo de eventos de la ventana de login.

Explicación del código de la placa Raspberry en Python:

A continuación, se explicara el código empleado en Python para la placa Raspberry Pi Pico W, haciendo uso de conjuntos de líneas para una mejor visualización. Las líneas en blanco han sido omitidas.

```
C:\> Users\ASUS\OneDrive\Escritorio > main.py
1  # En Raspberry Pi: Transmisión MJPEG por Flask
2  from flask import Flask, Response
3  from picamera2 import Picamera2
4  import cv2, time
5
6  app = Flask(__name__)
7  picam2 = Picamera2()
8  config = picam2.create_video_configuration(main={"format": "YUV420", "size": (640, 360)})
9  picam2.configure(config)
10 picam2.start()
11 time.sleep(1)
12
13 def generate():
14     while True:
15         frame = picam2.capture_array("main")
16         frame = cv2.cvtColor(frame, cv2.COLOR_YUV2BGR_I420)
17         jpeg = cv2.imencode('.jpg', frame)
18         yield (b'--frame\r\nContent-Type: image/jpeg\r\n\r\n' + jpeg.tobytes() + b'\r\n\r\n')
19
20 @app.route('/video')
21 def video_feed():
22     return Response(generate(), mimetype='multipart/x-mixed-replace; boundary=frame')
23
24 app.run(host="0.0.0.0", port=8080)
25
```

Líneas 1-24:

Línea 1: # En Raspberry Pi: Transmisión MJPEG por Flask

- Comentario descriptivo: indica que el script transmite video MJPEG usando Flask en una Raspberry Pi.

Línea 2: from flask import Flask, Response

- Flask: Microframework web de Python para crear servidores HTTP.
- Response: Clase de Flask que permite construir respuestas HTTP personalizadas, como la de video en vivo.

Línea 3: from picamera2 import Picamera2

- Picamera2: Módulo para controlar la cámara de la Raspberry Pi de forma avanzada.
- Picamera2: Clase principal para capturar imágenes o video.

Línea 4: import cv2, time

- cv2: Módulo de OpenCV para procesamiento de imágenes.
- time: Módulo estándar de Python para trabajar con temporizadores o pausas.

Línea 6: app = Flask(__name__)

- Crea una instancia de la aplicación Flask.

- `__name__` se usa para determinar si el script se está ejecutando directamente o es importado.

Línea 7: `picam2 = Picamera2()`

- Crea una instancia del controlador de la cámara Raspberry Pi usando la clase `Picamera2`.

Línea 8: `config = picam2.create_video_configuration(main={"format": "YUV420", "size": (640, 360)})`

- Configura la resolución y el formato de salida del video en el modo principal.
- YUV420: Formato de color eficiente en ancho de banda.
- (640, 360): Resolución del video.

Línea 9: `picam2.configure(config)`

- Aplica la configuración previamente creada a la cámara.

Línea 10: `picam2.start()`

- Inicia la cámara con la configuración actual.

Línea 11: `time.sleep(1)`

- Espera 1 segundo para asegurarse de que la cámara esté completamente inicializada antes de capturar imágenes.

Línea 13: `def generate():`

- Define una función generadora que produce un flujo continuo de imágenes JPEG para transmitir como MJPEG.

Línea 14: `while True:`

- Bucle infinito que permite transmitir video en tiempo real.

Línea 15: `frame = picam2.capture_array("main")`

- Captura un cuadro de video desde el stream principal de la cámara en formato YUV.

Línea 16: `frame = cv2.cvtColor(frame, cv2.COLOR_YUV2BGR_I420)`

- Convierte el cuadro capturado de YUV a BGR, que es el formato estándar en OpenCV.

Línea 17: `_, jpeg = cv2.imencode('.jpg', frame)`

- Codifica la imagen BGR en formato JPEG, que será enviado como parte del stream MJPEG.

Línea 18: `yield (b'--frame\r\nContent-Type: image/jpeg\r\n\r\n' + jpeg.tobytes() + b'\r\n')`

- `yield`: Retorna cada imagen codificada como parte de un stream HTTP multipart para clientes web.

Línea 20: @app.route('/video')

- Define una ruta en Flask (/video) que se activará cuando se acceda a esa URL.

Línea 21: def video_feed():

- Función asociada a la ruta /video, que será llamada por Flask para manejar la solicitud.

Línea 22: return Response(generate(), mimetype='multipart/x-mixed-replace; boundary=frame')

- Devuelve una respuesta HTTP con el stream MJPEG generado por la función generate().

Línea 24: app.run(host="0.0.0.0", port=8080)

- Inicia el servidor Flask accesible desde cualquier IP (0.0.0.0) en el puerto 8080. Ideal para acceso en red local.

REQUISITOS:

1. Requisitos de Hardware

Para el correcto funcionamiento del sistema de seguridad inteligente basado en detección visual remota con inteligencia artificial, se requiere el siguiente equipamiento mínimo:

- Raspberry Pi 5 de 8 GB de RAM, que actuará como nodo de captura y transmisión de video.
- Módulo de cámara compatible, preferentemente la Raspberry Pi Camera Module 5, con interfaz CSI para una mayor fluidez y compatibilidad.
- Tarjeta microSD de 32 GB, clase 10, con sistema operativo Raspberry Pi OS.
- Fuente de alimentación oficial para Raspberry Pi (5.1V/3A USB-C).
- MacBook o Laptop con sistema operativo macOS, Windows o Linux, que actuará como estación cliente para la visualización, análisis y gestión del sistema de seguridad.
- Conexión a Internet estable en ambos extremos (Raspberry y cliente), requerida para la transmisión del video mediante el servicio ngrok.

2. Requisitos de Software

A nivel de software, el sistema requiere la instalación y configuración de los siguientes componentes:

- Python 3.8 o superior, instalado tanto en la Raspberry Pi como en la estación cliente.
- Librerías de Python necesarias:
 - opencv-python
 - ultralytics (para YOLOv8)
 - Pillow
 - tkinter (incluido por defecto en la mayoría de instalaciones de Python)
 - twilio (para envío de alertas SMS)
 - python-dotenv (para manejo de credenciales)
- Ngrok para la exposición segura del stream de video capturado por la Raspberry Pi a través de un túnel HTTP público.
- Archivo .env configurado adecuadamente con las credenciales necesarias para:
 - Acceso SMTP del correo electrónico de envío de alertas
 - Credenciales de Twilio (SID, token, número)
- Archivo de configuración ngrok.yml, debidamente autenticado con el token de la cuenta y apuntando al puerto de transmisión del servidor de video.

- Modelo YOLOv8 (yolov8n.pt) descargado y disponible localmente.

Uso del Sistema

El sistema de seguridad inteligente consta de dos componentes principales: un servidor de captura y transmisión de video alojado en una Raspberry Pi, y una interfaz gráfica de monitoreo y análisis que se ejecuta en una computadora personal (cliente). A continuación, se describen los pasos de uso para la correcta operación del sistema.

1. Inicio del servidor de video (Raspberry Pi)

La Raspberry Pi utiliza el módulo Picamera2 y la biblioteca Flask para capturar imágenes en tiempo real desde su cámara integrada y transmitir las como un flujo MJPEG a través de la red. El archivo principal main.py contiene el siguiente proceso:

- Se inicializa la cámara usando Picamera2.
- Se configura la salida de video en formato YUV420 a una resolución de 640x360 píxeles.
- Se lanza un servidor Flask local que expone una ruta /video, donde se emite continuamente el flujo de imágenes capturadas, convertido al formato JPEG.
- El servidor queda escuchando en el puerto 8080, permitiendo el acceso desde cualquier dispositivo conectado a la red, o desde el exterior usando herramientas como ngrok.

2. Acceso al flujo de video desde el cliente

En la computadora del usuario (MacBook, PC u otro dispositivo), se ejecuta una interfaz gráfica basada en Tkinter, la cual accede al flujo de video emitido por la Raspberry Pi mediante su URL pública (obtenida con ngrok o una IP fija).

- El usuario inicia sesión en la interfaz.
- Se activa el monitoreo en vivo mediante el botón “Live”, que carga el stream remoto usando cv2.VideoCapture(URL).
- Se aplica detección de objetos en tiempo real mediante un modelo YOLOv8 cargado localmente. En este caso, se identifican personas y armas (knife, gun), clasificando visualmente a los individuos como:
 - Persona Agresiva: Si se encuentra cercana a un arma.
 - Víctima: Si está próxima a una persona agresiva pero no porta armas.
 - Persona: Si no hay amenazas en la escena.

Este procesamiento se muestra sobre la interfaz gráfica, junto a funciones adicionales como:

- Captura de imágenes.
- Análisis posterior de imágenes capturadas.
- Envío de alertas por sonido, SMS (Twilio) y correo electrónico (SMTP) ante la detección de situaciones potencialmente peligrosas.

3. Flujo resumido

- I. La Raspberry transmite video en vivo mediante /video.
- II. La PC del cliente accede a dicha URL.
- III. Se detectan objetos con YOLOv8 en tiempo real.
- IV. Se visualiza el estado de cada persona en pantalla.
- V. Si se detectan amenazas, se activa el sistema de alerta múltiple.

Configuración de Transmisión con Ngrok

Para permitir el acceso remoto al flujo de video generado por la Raspberry Pi, incluso desde redes externas o dinámicas, se ha integrado Ngrok como solución de túnel seguro. Ngrok permite exponer el puerto local donde se ejecuta el servidor Flask a una URL pública accesible desde cualquier lugar del mundo, sin necesidad de configurar el router ni abrir puertos manualmente.

1. Requisitos Previos

Antes de proceder con la configuración, asegúrese de que:

- Tiene una cuenta activa en Ngrok.
- Ha instalado ngrok en la Raspberry Pi. Si no lo ha hecho, puede instalarlo ejecutando:

```
wget https://bin.equinox.io/c/4VmDzA7iaHb/ngrok-stable-linux-arm.zip  
unzip ngrok-stable-linux-arm.zip  
sudo mv ngrok /usr/local/bin
```

2. Autenticación con su Cuenta Ngrok

Una vez instalado, es necesario vincular ngrok con su cuenta para habilitar túneles personalizados y persistentes. Inicie sesión en su cuenta de Ngrok y copie el Auth Token que aparece en el panel de control. Luego, configúrelo en la Raspberry ejecutando:

```
ngrok config add-authtoken TOKEN
```

Esto almacenará su token de autenticación en ~/.ngrok2/ngrok.yml.

3. Creación del Túnel para el Servidor Flask

El servidor Flask que ejecuta main.py se lanza por defecto en el puerto 8080. Para exponer este puerto, basta con ejecutar:

```
ngrok http 8080
```

Ngrok generará una URL pública como:

```
https://raspberrypbrowser.ngrok.app
```

4. Automatización al Iniciar la Raspberry Pi

Para facilitar la operación sin intervención manual, se recomienda automatizar el lanzamiento tanto de ngrok como del script main.py al iniciar la Raspberry Pi. Esto puede lograrse añadiendo las siguientes líneas al archivo ~/.bashrc, ~/.profile o como servicio systemd:

```
# Inicia el servidor Flask

python3 /home/pi/proyecto/main.py &

# Espera unos segundos y luego lanza Ngrok

sleep 5

ngrok http 8080 --log=stdout > /home/pi/ngrok.log &
```

También puede usarse un archivo ngrok.yml personalizado en ~/.ngrok2/ para definir dominios reservados o configuraciones más avanzadas.

5. Integración con la Interfaz Cliente

Una vez lanzado ngrok, la URL proporcionada debe ser utilizada como origen del flujo de video en la interfaz gráfica del cliente, por ejemplo:

```
url = 'https://raspberrypbrowser.ngrok.app/video'

cap = cv2.VideoCapture(url)
```

Esta URL puede cambiar cada vez que se reinicia ngrok, a menos que utilice una cuenta Ngrok Pro con subdominios personalizados. Para entornos de producción, se recomienda usar un subdominio fijo con el siguiente comando:

```
ngrok http --domain=raspberrybrowser.ngrok.app 8080
```

Esto requiere tener configurado dicho subdominio previamente en la cuenta de Ngrok.

Referencias bibliograficas:

Alpha2lucifer. (s.f.). *sentinalsecure*. GitHub.
<https://github.com/alpha2lucifer/sentinalsecure>

Alexisvalentino. (s.f.). *Threat-Detection-System*. GitHub.
<https://github.com/alexisvalentino/Threat-Detection-System>

Ciscoblockchain. (s.f.). *YOLO-Pi*. GitHub. <https://github.com/CiscoBlockChain/YOLO-Pi>

Shruthika-s. (s.f.). *VisionGuard: Anomaly Detection in CCTV footage*. GitHub.
<http://github.com/Shruthika-s/VisionGuard-Anomaly-Detection-in-CCTV-footage>

Yigitekin. (s.f.). *Human Detection using YOLOv5 and Raspberry Pi*. GitHub.
<https://github.com/YigitEkin/Human-Detection-using-YOLOv5-and-Raspberry-Pi>